This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

Detecting LLM Fact-conflicting Hallucinations Enhanced by Temporal-logic-based Reasoning

Ningke Li*, Yahui Song*, Yuekang Li, Ling Shi, Yi Liu, Haoyu Wang, Kailong Wang†

Abstract—Large language models (LLMs) face the challenge of hallucinations – outputs that seem coherent but are actually incorrect. A particularly damaging type is fact-conflicting hallucination (FCH), where generated content contradicts established facts. Addressing FCH presents three main challenges: **1)** Automatically constructing and maintaining large-scale benchmark datasets is difficult and resource-intensive; **2)** Generating complex and efficient test cases that the LLM has not been trained on – especially those involving intricate temporal features – is challenging, yet crucial for detecting hallucinations; and **3)** Validating the reasoning behind LLM outputs is inherently difficult, particularly with complex logical relationships, as it requires transparency in the model's decision-making process. This paper presents DROWZEE, an innovative end-to-end metamorphic testing framework that utilizes temporal logic to identify fact-conflicting hallucinations (FCH) in large language models (LLMs). DROWZEE builds a comprehensive factual knowledge base by crawling sources like Wikipedia and uses automated temporal-logic reasoning to convert this knowledge into a large, extensible set of test cases with ground truth answers. LLMs are tested using these cases through template-based prompts, which require them to generate both answers and reasoning steps. To validate the reasoning, we propose two semantic-aware oracles that compare the semantic structure of LLM outputs to the ground truths. Across nine LLMs in nine different knowledge domains, experimental results show that DROWZEE effectively identifies rates of non-temporal-related hallucinations ranging from 24.7% to 59.8%, and rates of temporal-related hallucinations ranging from 16.7% to 39.2%. Key insights reveal that LLMs struggle with out-of-distribution knowledge and logical reasoning. These findings highlight the importance of continued efforts to detect and mitigate hallucinations in LLMs.

Index Terms—Large Language Model, Hallucination, Temporal Logic, Metamorphic Testing

1 INTRODUCTION

Large Language Models (LLMs) have revolutionized language processing, demonstrating impressive text generation and comprehension capabilities with diverse applications. However, despite their growing use, LLMs face significant security and privacy challenges [1], [2], [3], which affect their overall effectiveness and reliability. A critical issue is the phenomenon of *hallucination*, where LLMs generate outputs that are coherent but factually incorrect or irrelevant. This tendency to produce misleading information compromises the safety and usability of LLM-based systems. This paper focuses on *fact-conflicting hallucina*tion (FCH), the most prominent form of hallucination in LLMs. FCH occurs when LLMs generate content that directly contradicts established facts. For instance, as illustrated in Fig. 1, an LLM incorrectly asserts that "*Haruki Murakami won the Nobel Prize in Literature in 2016*", whereas the fact is that "*Haruki Murakami has not won the Nobel Prize, though he has received numerous other literary awards*". Such inaccuracies can significantly lead to user confusion and undermine the trust and reliability that are crucial for LLM applications.



Fig. 1: A Hallucination Output Example

Recent studies have introduced various methods to detect LLM hallucinations. A common approach involves developing specialized benchmarks, such as TruthfulQA [4], HaluEval [5], and KoLA [6], to assess hallucinations in tasks like question-answering, summarization, and knowledge graphs. While manually labeled datasets provide valuable insights, current methods often rely on simplistic or semi-automated techniques such as string matching, manual validation, or verification through another language model. These approaches reveal significant gaps in automatically and effectively detecting fact-conflicting hallucinations (FCH). The primary challenges in FCH detection arise from the lack of dedicated ground truth datasets, the absence of comprehensive test cases designed to trigger

N. Li is with Huazhong University of Science and Technology, China, and National University of Singapore, Singapore. E-mail: ningke_li@u.nus.edu
Y. Song and K. Wang are with National University of Singapore, Singapore. E-mail: yahui.song@u.nus.edu, wangkl@nus.edu.sg
Y. Li is with the University of New South Wales, Australia. E-mail: yuekang.li@unsw.edu.au
L. Shi is with AIDX TECH PTE. LTD., Singapore. E-mail: ling.shi@aidxtech.com
Y. Liu is with Griffith University, Australia. E-mail: yi.liu@griffith.edu.au
H. Wang is with Huazhong University of Science and Technology, China. E-mail: haoyuwang@hust.edu.cn
⋆ *Ningke Li and Yahui Song contribute equally to this work.*
† *Kailong Wang is the corresponding author.*

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

FCH, and the lack of a robust testing framework. Unlike other types of hallucinations, such as input-conflicting or context-conflicting hallucinations [7], [8], which can often be identified through semantic consistency checks, detecting FCH requires the verification of factual accuracy against external knowledge sources/databases. This process is particularly challenging and resource-intensive, especially for tasks that involve complex logical relationships. We identify three primary challenges in addressing this research gap:

1) **Automatically constructing and updating benchmark datasets.** Existing methodologies mainly rely on manually curated benchmarks for detecting specific hallucinations, which fail to encompass the broad and dynamic spectrum of fact-conflicting scenarios in LLMs. Meanwhile, due to the ever-evolving nature of knowledge, the need for frequent updates to benchmark data imposes a substantial and continuous maintenance effort. The reliance on benchmark datasets thus restricts the FCH detection techniques' adaptability, scalability, and detection capability;

2) **Efficiently generating FCH test cases.** LLMs often answer correctly to simple, straightforward questions due to their extensive training on vast datasets. However, to effectively assess their reasoning capabilities, it is important to generate more complex questions, such as those involving intricate temporal characteristics, that require reasoning rather than just recalling facts. However, constructing such test cases is non-trivial. The challenge lies in designing questions that use familiar knowledge but involve reasoning patterns the LLM may not have been explicitly trained on. Creating such test cases efficiently while ensuring they probe reasoning skills in ways the model has not previously encountered is essential to uncovering latent hallucinations;

3) **Validating the reasoning steps from LLM outputs.** Even when LLMs finally produce correct answers, the outputs may not indicate an accurate reasoning process, potentially masking false understanding – a source of FCH. Additionally, the quality of manual validation can differ based on human expertise. As a result, automatically validating reasoning processes, particularly those involving complex logical relationships, is inherently challenging.

To address the problems outlined above, this paper presents a novel automatic end-to-end metamorphic testing technique based on temporal logic for detecting FCH. To the best of our knowledge, we are the first to create a comprehensive FCH testing framework that utilizes factual knowledge reasoning and metamorphic testing, all seamlessly integrated into the fully automated tool, DROWZEE.

DROWZEE begins by establishing a comprehensive factual knowledge base sourced through crawling information from accessible knowledge bases such as Wikipedia. Each piece of this knowledge acts as a "seed" for subsequent transformations. Leveraging logical operators to automatically generate temporal reasoning rules, we transform and augment these seeds and expand factual knowledge into a well-established set of question-answer pairs. Using the questions and answers in the knowledge set as test cases and ground truth, respectively, we construct a reliable and robust FCH testing benchmark.

The experiment uses a series of carefully designed template-based prompts to test for FCHs in LLMs. To thoroughly evaluate the reasoning behind the responses, we instruct the LLMs not only to generate answers to the test cases but also to provide detailed justifications for their answers. To reliably identify FCH, we introduce two semantic-aware, similarity-based metamorphic oracles. These oracles extract the key semantic elements from each sentence and map out the logical relationships between them. By comparing the logical and semantic structures of the LLM's responses with the ground truth, the oracles can detect FCH by identifying significant deviations in the LLM's answers from the correct information.

We demonstrate the effectiveness of our approach through comprehensive experiments in multiple contexts. First, our evaluation involves deploying DROWZEE across a wide range of topics drawn from a diverse selection of Wikipedia articles. Second, we test our framework on various open-source and commercial LLMs, thoroughly assessing its applicability and performance across different model architectures. Our key findings indicate that DROWZEE succeeds in automatically generating practical test cases and identifying hallucination issues of nine LLMs across nine domains. Using these test sets, our experiments show that the rate of hallucination responses produced by various LLMs ranges from 24.7% to 59.8% for cases unrelated to temporal reasoning and 16.7% to 39.2% for cases requiring temporal reasoning. We then categorize these hallucination responses into *erroneous knowledge hallucination* and *erroneous inference hallucination*. Through an in-depth analysis, we unveil that the lack of logical reasoning capabilities contributes the most to the FCH issues in LLMs. Additionally, we observe that LLMs are particularly prone to generating hallucinations in test cases involving temporal concepts and out-of-distribution knowledge. Such an evaluation demonstrates that the test cases generated using logical reasoning rules can effectively trigger and detect LLM hallucinations.

This paper builds upon the earlier version [9] by incorporating hallucination detection through temporal-logic-guided test generation. It includes additional motivational examples (§2), a comprehensive set of reasoning rules for encoding *Metric Temporal Logic* (MTL) [10] formulae (§3.4) and automatically generating temporal-logic-related question-answer pairs (§3.5), and further experimental studies (the RQ4 at §4) that detect hallucinations due to insufficient temporal reasoning capabilities. The main contributions of this work are summarized as follows:

- **A novel FCH testing framework.** To the best of our knowledge, we are the first to develop a novel testing framework based on logic programming and metamorphic testing to automatically detect FCH issues in LLMs.
- **An extensive benchmark based on factual knowledge.** To facilitate collaborative efforts and future advances in identifying FCH, the source code of DROWZEE and the constructed benchmark dataset are publicly available.[1]
- **Test generation via temporal reasoning.** Our tool automatically generates test cases that provide a more comprehensive evaluation of LLMs in handling reasoning tasks and identifying factual inconsistencies. By applying temporal logic-based reasoning rules, we expand the

[1] https://github.com/security-pride/Drowzee

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

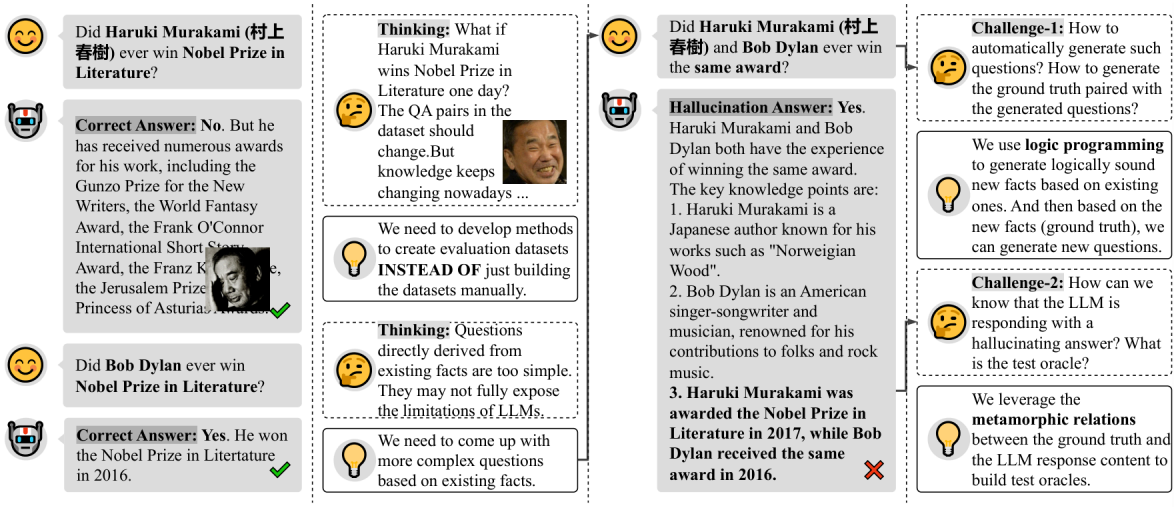


Fig. 2: Motivating Examples for Automatic Benchmark Construction with Complex Questions

initial seed data from our knowledge base, enhancing the diversity and complexity of the test scenarios.

- **Semantic-aware oracles for LLM answer validation.** We propose and implement two automated verification mechanisms, i.e., the oracles, that analyze the semantic structure similarity between sentences. These oracles are designed to validate the reasoning logic behind the answers generated by LLMs, hereby reliably detecting the occurrence of FCHs.

2 Motivating Examples

2.1 Automatic Benchmark Construction

As a first motivating example, shown in Fig. 2, given the facts about whether Haruki Murakami and Bob Dylan have won the Nobel Prize, as illustrated in the left sub-figure, we can query straightforward questions such as "*whether Haruki Murakami or Bob Dylan has won the Nobel Prize?*". Asking and verifying this knowledge requires no logical reasoning. However, such questions are often not enough to unveil hallucinations. Therefore, more diversified questions, i.e., with intertwined and complex information, as illustrated in the right sub-figure, are needed. Moreover, the knowledge landscape is dynamic, with new information continuously surfacing and older information becoming obsolete. If facts change continuously over time, for instance, if Haruki Murakami were to win the Nobel Prize in the future, this would necessitate regular updates and corrections to the ground truth in existing datasets to reflect them. However, maintaining the accuracy of these benchmarks requires a significant amount of manual labor.

2.2 Questions Involving Temporal Reasoning

As LLMs increasingly rely on temporal reasoning to process time-dependent data, understanding how well they can handle temporal logic is crucial for their development and deployment in real-world applications. Reasoning about temporal-logic-related queries often requires complex steps, particularly regarding the relative timing of multiple events. Unfortunately, manually generated test cases frequently lack thorough completeness verification, which undermines their reliability. We demonstrate such an example in Table 1. The question pertains to the relative timing relationship between the original Ben_10 TV series and the year 2000. It is known that the original Ben_10 series premiered on December 27, 2005, and concluded in 2008. Therefore, when considering the query asking whether "Ben_10 finally airs within the first three years after 2000", the expected answer is "No", as none of the years from 2005 to 2008 fall within that queried timeframe. However, despite the accurate facts inputs, LLMs still generate hallucinations due to insufficient temporal reasoning.

Query: Did the Original Series of Ben_10 TV show finally air within the time frame of the 1st year to the 3rd year after the year 2000? Please answer Yes or No and explain why.
Expected Answer: No. The series started in 2005. The period from the 1st to 3rd year after 2000 is [2001, 2003]. Since 2005 is not within the interval [2001, 2003], the answer is No.
Hallucinations Answer (Claude 3.5): Yes, the original series of Ben_10 aired within the time frame of the 1st year to the 3rd year after the year 2000. The show premiered on December 27, 2005, and concluded its run on April 15, 2008. This timeline clearly falls within the specified period from 2001 to 2003.
Analysis: The LLM correctly recalls the start date (2005) but fails the temporal reasoning. It incorrectly concludes that 2005 is within the [2001, 2003] interval, a clear logical contradiction that constitutes a fact-conflicting hallucination.

TABLE 1: Temporal Logic Reasoning Hallucination

In this work, our proposed testing framework automatically generates such temporal test cases in the form of MTL formulae, denoted by ϕ, which incorporate quantitative timing constraints and enable the expression of temporal relationships with precise intervals. For example, this query shown in Table 1 is represented as $\phi = \mathcal{F}_{[1,3]}(Ben_10)$, where $\mathcal{F}$ stands for *finally* and [1,3] is the time frame. This formula queries all time points t' for which the event Ben_10 finally happened within 1 to 3 years after t'. The ground truth interval I that satisfies ϕ is $[2002, 2007]$, which

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191


```
// Ground Facts crawled from Wikipedia
begin('Ben_10',2005). end('Ben_10',2008).

// Generating the time-stamped fact
ben_10(Start,End) :- Start=<End,
    begin('Ben_10',Start), end('Ben_10',End).

// Encding the MTL formula φ
finally_ben_10_during_1_3(Start,End) :-
    ben_10(Start1,End1),
    Start is (Start1-3), End is (End1-1).

// The time interval I which satisfies φ
?- finally_ben_10_during_1_3(Start,End).
   Start = 2002, End = 2007.
```

Fig. 3: Prolog Encoding for $\phi = \mathcal{F}_{[1,3]}(Ben_10)$

is derived from the event's known interval $[2005, 2008]$. The formula $\mathcal{F}_{[1,3]}$ is satisfied at a time t' if the event starts between $t'+1$ and $t'+3$. At t'=2002, the future window is $[2003, 2005]$. Since Ben_10 starts in 2005 (the last year of this window), the event finally happens within the window. For any earlier t' (e.g., 2001 with window $[2002, 2004]$), the start year 2005 falls outside the window. At t'=2007, the future window is $[2008, 2010]$. Although Ben_10 started in 2005 (before this window), it was still ongoing in 2008. Therefore, for someone at time t'=2007 looking forward, Ben_10 will finally be happening/have happened during the $[2008, 2010]$ window. For t'=2008, the future window is $[2009, 2011]$, and since Ben_10 ended in 2008, it would no longer be happening in this future window.

To automatically obtain the ground truth interval I, DROWZEE automatically generates the Prolog encoding rules for ϕ, as shown in Fig. 3. These rules accurately determine that $I = [2002, 2007]$, indicating that all time points within this interval satisfy ϕ. Then, the validity of 2000 is easily disproved because 2000 is not a member of I.

It's important to note that such reasoning rules can be generated for arbitrarily nested MTL formulae. These rules lead to sound and deterministic conclusions regarding a "Yes" or "No" response, and they provide reasoning steps to assist in evaluating the LLMs' answers, especially when the queries become more complex. Furthermore, obtaining the ground truth interval directly enables us to flexibly control the generation of both positive and negative test cases.

3 METHODOLOGY

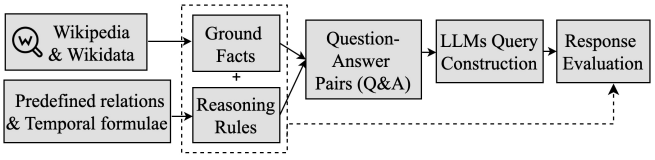


Fig. 4: DROWZEE Overview

DROWZEE (Fig. 4) is a general-purpose testing framework that evaluates the LLM outputs for automatically generated test cases. The inputs for the response evaluation contain a natural language (NL) query for LLM and its ground truth answer obtained using logic programming (§3.1). Based on voluminous knowledge database dumps, DROWZEE extracts factual knowledge (§3.2), which outputs a set of predicates in the form of Prolog facts. Then, DROWZEE deploys a set of pre-defined or automatically generated reasoning rules to extend the database with a set of derived facts (§3.3, §3.4). These derived facts facilitate an automated test generation (§3.5), which outputs question-answer pairs (Q&A) and concrete prompts for LLMs. Given the Q&A pairs and the LLM outputs, DROWZEE evaluates the responses from LLMs and detects factual inconsistency automatically (§3.6). To this end, it first parses LLM outputs into a semantic-aware structure, and evaluates their semantic similarity to the ground truth. Lastly, it develops similarity-based oracles that apply metamorphic testing to assess consistency against the ground truth.

3.1 Preliminary

$$
\begin{array}{lrcl}
(Program) & \mathcal{P} & ::= & \widetilde{R} \mathbin{+\!\!+} \widetilde{Q} \\
(Rule) & Q & ::= & R \text{ :- } \widetilde{body} \\
(Body) & body & ::= & \texttt{Pos}\ R \mid \texttt{Neg}\ R \mid \pi \\
(Predicate) & R & ::= & nm\,(\widetilde{entity}) \\
(Pure) & \pi & ::= & T \mid F \mid bop(t_1, t_2) \mid \pi_1 \wedge \pi_2 \mid \pi_1 \vee \pi_2 \mid \neg\pi \\
(Term) & t & ::= & c \mid X \mid t_1 + t_2 \mid t_1 - t_2
\end{array}
$$

Fig. 5: A Core Syntax of Prolog

Logic programming allows the programmer to specify the rules and facts, enabling the Prolog interpreter to infer answers to the given queries automatically. We define a core syntax of Prolog in Fig. 5. A Prolog program consists of two parts: a set of facts ($\widetilde{R}$) and a set of rules ($\widetilde{Q}$). Throughout the paper, we use the over-tilde notation to denote a set of items. For example, $\widetilde{X}$ refers to a set of variables, i.e., $\{X_1, \ldots, X_n\}$. A fact is represented as a relational predicate with a name and a set of entity arguments, where nm is an arbitrary distinct identifier drawn from a finite set of relations. Entities are drawn from the knowledge database, ranging from string types (for names or events) and integers (for time points). A Prolog rule is a Horn clause that comprises a head predicate and a set of body predicates placed on the left and right sides of the arrow symbol (:–).

A rule means that the left-hand side is logically implied by the right-hand side. The rule bodies are either positive or negative relations, corresponding to the requirements upon the presence or absence of facts. We use "R" and "$!\,R$" as abbreviations for "`Pos` R" and "`Neg` R", respectively. Rule bodies contain pure formulae and simplified and decidable sets of Presburger arithmetic predicates over local variables. The Boolean values of *true* and *false* are respectively indicated by T and F. The binary operators bop are from $\{<, \leq, =, \geq, >\}$. Terms consist of constants (denoted by c), program variables (denoted by X), or simple computations of terms, such as t_1+t_2 and t_1-t_2.

3.2 Factual Knowledge Extraction

To facilitate an automated reasoning system, we extract the *ground facts* in the structure of three-element predicates, i.e., $nm\,(s, o)$, where "s" (stands for *subject*) and "o" (stands for *object*) are entities, and "nm" is the name of the predicate.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

Here, we follow the convention of Prolog, where variable names must start with an uppercase letter, and any name that begins with a lowercase letter is a constant.

Existing knowledge databases [11], [12], [13], [14] not only encompass a vast array of documents but also provide structured data, facilitating an ideal source for constructing a rich factual knowledge base. Thus, the genesis of our test cases is exclusively rooted in the entities and structured relations sourced from existing knowledge databases, ensuring a sophisticated and well-informed foundation for our testing framework. Specifically, we follow the categorization for entities (Fig. 6) and relations (Fig. 7) used by WikiPedia [12] to perform a thorough facts extraction. In particular, the **Prop.** (stands for properties) entry for relations guides the automated generation of reasoning rules detailed in §3.3.

The facts extraction is done per-category basis, implementing a divide-and-conquer strategy, which efficiently integrates all the facts from all the categories. As shown in Algorithm 1, for any given entity category and relation category, the function ExtractGroundFacts iterates through all possible entities and relations. For each combination $(entity, nm)$, it queries the database using the QueryDB function, which retrieves all three-element facts established with the specific predicate nm and the argument $entity$ placed either in the subject or the object position.

Algorithm 1 Facts Extraction

Require: Entity Category (EC), Relation Category (RC)
Ensure: Ground Facts ($\widetilde{R}_{ground}$)
1: **function** ExtractGroundFacts(EC, RC)
2: $\widetilde{R}_{ground} \leftarrow []$ ▷ Initialization
3: **for** $entity \in EC$ **do** ▷ Iterate over each entity
4: **for** $nm \in RC$ **do** ▷ Iterate over each relation
5: $\widetilde{R} \leftarrow$ QueryDB($entity$, nm) ▷ Retrieve ground facts
6: $\widetilde{R}_{ground}.append(\widetilde{R})$ ▷ Extend the ground facts
7: **return** $\widetilde{R}_{ground}$ ▷ Return the ground facts

Entity Cat.	Description
Culture and the Arts	Famous films, books, etc.
Geography and Places	Countries, cities and locations.
Health and Fitness	Diseases and genes.
History and Events	Famous historical events, etc.
People and Self	Important figures.
Mathematics and Logic	Formulas and theorems.
Natural and Physical Sciences	Celestial bodies and astronomy.
Society and Social Sciences	Major social institutions, etc.
Technology and Applied Sciences	Computer science, etc.

Fig. 6: Entity Categorization.

Relation Cat.	Examples	Prop.
Noun Phrase	*place_of_birth (barack_obama, honolulu).* *genre (28_days_later, horror_film).*	[R-Neg] [R-Sym] [R-Trans]
Verb Phrase (Passive Voice)	*killed_by (alexander_pushkin, georges-charles_de_heeckeren_d'anthès).* *located_in_time_zone (arizona, utc-07:00).*	[R-Neg] [R-Inverse]
Verb Phrase (Active Voice)	*follows (4769_Castalia, 4768_hartley).* *replaces (american_broadcasting_company, nbc_blue_network).*	[R-Neg] [R-Inverse]

Fig. 7: Relation Categorization.

3.3 Deriving Simple Facts via Logical Reasoning

Based on the relation category, each predicate is labeled with a different set of properties, shown in Fig. 7, which are mapped to different derivation rules.

Based on the ground facts extracted from the databases, DROWZEE derives additional facts to enrich the knowledge and generates test cases from each derived fact. As shown in Fig. 8, it provides four basic derivation rules, providing sound strategies to generate mutated facts from the ground facts. Note that these rules are also prevalently adopted in several literature [15], [16], [17], [18], [19] in the context of knowledge reasoning. Given a predicate name nm, the derivation "$nm \hookrightarrow (nm_{new}, Q)$" holds if nm can be applied into a Prolog rule Q, and produces more facts upon a new predicate with the name nm_{new}. These new predicates are freshly generated based on predefined suffixes.

As indicated in Fig. 7, all the predicates can be applied to the $[R\text{-}Neg]$ rule, which derives the negated relations, e.g., "$!was$" using its positive counterpart, e.g., "$wasn't$". For the *noun phrase* relations, both $[R\text{-}Sym]$ and $[R\text{-}Trans]$ rules can apply, which generate more facts without creating new predicates. For the *verb phrase* relations, both passive voice and active voice predicates can be applied to the $[R\text{-}Inverse]$ rule, which captures the inverse relations, where the subject and object can be reversely linked through a variant of the original relation. An example of such a rule is:

$$influence(O, S) :\!- influence_by(S, O).$$

We summarize the fact derivation process using Algorithm 2. Given any relation category, we iterate its predicates and generate the derivation rule Q (Line 4). For simplicity, we assume that the choice of which derivation rule to apply is predetermined. Based on this assumption, a new Prolog program is constructed, comprising ground facts and Q. In particular, we use $[\![R]\!]_{\mathcal{P}}$ to denote the query results of R concerning the Prolog program $\mathcal{P}$, and $\mathcal{P}$ contains all the

$$\frac{Q = nm_{new}(S, O) :\!- !nm(S, O)}{nm \hookrightarrow (nm_{new}, Q)}\ [R\text{-}Neg] \qquad \frac{Q = nm_{new}(S, O) :\!- nm(O, S)}{nm \hookrightarrow (nm_{new}, Q)}\ [R\text{-}Inverse]$$

$$\frac{Q = nm(S, O) :\!- nm(O, S)}{nm \hookrightarrow (nm, Q)}\ [R\text{-}Sym] \qquad \frac{Q = nm(S, O') :\!- nm(S, O), nm(O, O')}{nm \hookrightarrow (nm, Q)}\ [R\text{-}Trans]$$

Fig. 8: Deriving New Facts From the Known Facts

Algorithm 2 Deriving New Facts

Require: Ground Facts ($\widetilde{R}_{ground}$), Relation Category (RC)
Ensure: Derived Facts ($\widetilde{R}_{derived}$)
1: **function** DerivingFacts($\widetilde{R}_{ground}$, RC)
2: $\widetilde{R}_{derived} \leftarrow []$ ▷ Initialization
3: **for** nm in RC **do** ▷ Iterate each predicate
4: $nm \hookrightarrow (nm_{new}, Q)$ ▷ Obtain the new predicate
5: $\mathcal{P} \leftarrow \widetilde{R}_{ground}$++$Q$ ▷ Construct the prolog program
6: $instantiations \leftarrow [\![nm_{new}(S, O)]\!]_{\mathcal{P}}$
7: **for** (s, o) in $instantiations$ **do** ▷ Iterate each entity tuple
8: $R_{new} \leftarrow nm_{new}(s, o)$ ▷ Construct the derived fact
9: $\widetilde{R}_{derived}.append(R_{new})$ ▷ Append the derived facts
10: **return** $\widetilde{R}_{derived}$ ▷ Return the derived facts

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

ground facts and the derivation rule. Note that when R contains no variables, it returns Boolean results indicating the presence of the fact; otherwise, it outputs all the possible instantiation of the variables. For each instantiation that contains one subject "s" and one object "o", we then compose them with the new predicate, which is taken as a *derived fact*. These derived facts are later used to generate NL test cases, detailed in §3.5.

3.4 Deriving Facts via Temporal Reasoning

$$\begin{array}{rrcl} (MTL) & \phi & ::= & nm \mid \mathcal{F}_I\,\phi \mid \mathcal{G}_I\,\phi \mid \phi_1\,\mathcal{U}_I\,\phi_2 \mid \mathcal{N}\,\phi \mid \\ & & & \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \neg\phi \\ (Time\ Interval) & I & ::= & [n_{start}, n_{end}] \end{array}$$

Fig. 9: A Core Syntax of MTL

Apart from the basic derivation rules, DROWZEE can also automatically derive complex composition rules based on *Metric Temporal Logic* (MTL) [10]. Specifically, we generate temporal test cases based on randomly generated MTL formulae over historical events. We define the syntax for MTL formula in Fig. 9, which contains the temporal operators for "finally ($\mathcal{F}$)", "globally ($\mathcal{G}$)", "until ($\mathcal{U}$)", and "next ($\mathcal{N}$)". The atomic propositions here are basic event relations nm. The time intervals are pairs of natural numbers indicating the lower and upper bounds of the years; and the constraint $n_{start} \leq n_{end}$ is enforced implicitly for all time intervals. In this paper for simplicity, we use discrete time measured in years as the smallest time interval. However, the framework can be extended to accommodate any smaller discrete time intervals, such as days or seconds.

Definition 1 (A Point-based Semantics for MTL)**.** *Given a set of (historical) facts $\mathcal{H}$, recording all the events that happened in history, an MTL formula ϕ, and a concrete time point t, the satisfaction relation $(\mathcal{H}, t) \models \phi$ (read at the time point t, the history $\mathcal{H}$ satisfies ϕ) is recursively defined as follows:*

$$\begin{array}{ll} (\mathcal{H},t) \models nm & \textit{iff } \exists\, I.\ [\![nm_TS(I)]\!]_{\mathcal{H}} = \textit{true and } t \in I \\ (\mathcal{H},t) \models \mathcal{F}_I\,\phi & \textit{iff } \exists\, d.\ d \in I \textit{ and } (\mathcal{H}, t{+}d) \models \phi \\ (\mathcal{H},t) \models \mathcal{G}_I\,\phi & \textit{iff } \forall\, d.\ d \in I \textit{ and } (\mathcal{H}, t{+}d) \models \phi \\ (\mathcal{H},t) \models \mathcal{N}\,\phi & \textit{iff } (\mathcal{H}, t{+}1) \models \phi \\ (\mathcal{H},t) \models \neg\phi & \textit{iff } (\mathcal{H}, t) \not\models \phi \\ (\mathcal{H},t) \models \phi_1\,\mathcal{U}_I\,\phi_2 & \textit{iff } \exists\, d.\ d \in I \textit{ and } (\mathcal{H}, t{+}d) \models \phi_2 \textit{ and} \\ & \quad \forall\, k \textit{ with } t{<}k{<}(t{+}d), (\mathcal{H}, k) \models \phi_1 \\ (\mathcal{H},t) \models \phi_1 \wedge \phi_2 & \textit{iff } (\mathcal{H}, t) \models \phi_1 \textit{ and } (\mathcal{H}, t) \models \phi_2 \\ (\mathcal{H},t) \models \phi_1 \vee \phi_2 & \textit{iff } (\mathcal{H}, t) \models \phi_1 \textit{ or } (\mathcal{H}, t) \models \phi_2 \end{array}$$

To facilitate the generation of temporal-based Q&A pairs, we define the semantics model for the MTL formulae in Definition 1, where the history is a set of facts. Here, we use $\mathcal{H}$ as a set of historical relations, e.g., "$nm_TS(I, s, o)$", which are the time-stamped version relations of the three-element relations "$nm(s, o)$", derived by one of the following rules:

$$nm_TS(I,s,o) \text{:-} nm(s,o), start(o,n_1), end(o,n_2), I{=}[n_1,n_2].$$
$$nm_TS(I,s,o) \text{:-} nm(s,o), start(s,n_1), end(s,n_2), I{=}[n_1,n_2].$$

which construct the event intervals using the time stamps associated with the object or the subject, respectively. The "*start*" and "*end*" predicates are originally generated from the knowledge database and mark the starting and ending points of the duration of the object (or subject) event. For simplicity, we use "$nm_TS(I)$" to abbreviate "$nm_TS(I, s, o)$" when s and o are unambiguously unique from the context. We also use $[\![nm_TS(I)]\!]_{\mathcal{H}}$ to denote the validity of querying the presence of a fact $nm_TS(I)$ against the historical facts $\mathcal{H}$, which stores all the time-stamped three-element predicates.

We randomly generate temporal test cases based on the rich set of historical events and the syntax templates defined in Fig. 9. Each temporal question consists of a concrete MTL formula and a concrete time point, i.e., (ϕ, t). For example, the query "*At 1800, does Victorian era finally come within 40 years?*" is represented as $(\mathcal{F}_{[0,40]}\, victorian_era, 1800)$. Next, we show how to obtain the expected answer by automatically generating Prolog reasoning rules.

Given a query ϕ, the relation "$\phi \rightsquigarrow (nm, \widetilde{Q})$" holds if ϕ can be translated into a set of Prolog rules, i.e., $\widetilde{Q}$. Querying "$nm(I)$" $\widetilde{Q}$, against the known database facts yields a set of instantiation of the interval I. The validity of ϕ at any given time point t is then indicated by the existence of a concrete interval I such that $t \in I$. We define the full set of encoding rules for MTL operators in Fig. 10.

$$\frac{\widetilde{Q} = [nm_{new}(I) \text{:-} nm_TS(I).]}{nm \rightsquigarrow (nm_{new}, \widetilde{Q})}\ [R\text{-}AP]$$

$$\frac{\widetilde{Q}{=}[nm_{new}([I'_{start}\text{-}I_{end}, I'_{end}\text{-}I_{start}]) \text{:-} nm(I').]}{\mathcal{F}_I\,\phi \rightsquigarrow (nm_{new}, \widetilde{Q})}\ [R\text{-}Finally]$$

$$\frac{\widetilde{Q}{=}[nm_{new}([I'_{start}\text{-}I_{start}, I'_{end}\text{-}I_{end}]) \text{:-} nm(I').]}{\mathcal{G}_I\,\phi \rightsquigarrow (nm_{new}, \widetilde{Q})}\ [R\text{-}Globally]$$

$$\frac{\widetilde{Q}{=}[nm_{new}([I'_{start}\text{-}1, I'_{end}\text{-}1]) \text{:-} nm(I').]}{\mathcal{N}\,\phi \rightsquigarrow (nm_{new}, \widetilde{Q})}\ [R\text{-}Next]$$

$$\frac{\begin{array}{c}\phi_1 \rightsquigarrow (nm_1, \widetilde{Q}_1) \qquad \phi_2 \rightsquigarrow (nm_2, \widetilde{Q}_2) \\ \widetilde{Q}_3{=}[helper1([I'_{start}{+}I_{start}, I'_{end}{+}1]) \text{:-} nm_1(I').] \\ \widetilde{Q}_4{=}[helper2(I_1 \cap I_2) \text{:-} helper1(I_1), nm_2(I_2).] \\ \mathcal{F}_I\,(helper2) \rightsquigarrow (nm_f, \widetilde{Q}_5) \\ \widetilde{Q}_6{=}[nm_{new}(I_1 \cap I_2) \text{:-} nm_1(I_1), nm_f(I_2).]\end{array}}{\phi_1\,\mathcal{U}_I\,\phi_2 \rightsquigarrow (nm_{new}, \widetilde{Q}_1 \cup \widetilde{Q}_2 \cup \widetilde{Q}_3 \cup \widetilde{Q}_4 \cup \widetilde{Q}_5 \cup \widetilde{Q}_6)}\ [R\text{-}Until]$$

$$\frac{\begin{array}{c}\phi \rightsquigarrow (nm, \widetilde{Q}_1) \\ \widetilde{Q}{=}[nm_{new}(I) \text{:-} findall(I_1, nm), compl(I_1, I).]\end{array}}{\neg\phi \rightsquigarrow (nm_{new}, \widetilde{Q}_1 \cup \widetilde{Q})}\ [R\text{-}Neg]$$

$$\frac{\begin{array}{c}[R\text{-}Conj] \\ \phi_1 \rightsquigarrow (nm_1, \widetilde{Q}_1) \qquad \phi_2 \rightsquigarrow (nm_1, \widetilde{Q}_2) \\ \widetilde{Q}{=}[nm_{new}(I_1 \cap I_2) \text{:-} findall(I_1, nm_1), findall(I_2, nm_2)]\end{array}}{\phi_1 \wedge \phi_2 \rightsquigarrow (nm_{new}, \widetilde{Q}_1 \cup \widetilde{Q}_2 \cup \widetilde{Q})}$$

$$\frac{\begin{array}{c}[R\text{-}Disj] \\ \phi_1 \rightsquigarrow (nm_1, \widetilde{Q}_1) \qquad \phi_2 \rightsquigarrow (nm_1, \widetilde{Q}_2) \\ \widetilde{Q}{=}[nm_{new}(I_1 \cup I_2) \text{:-} findall(I_1, nm_1), findall(I_2, nm_2)]\end{array}}{\phi_1 \vee \phi_2 \rightsquigarrow (nm_{new}, \widetilde{Q}_1 \cup \widetilde{Q}_2 \cup \widetilde{Q})}$$

Fig. 10: Encoding MTL Formula ϕ using Prolog Rules

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

These encoding rules deploy several auxiliary predicates: the "$findall(I, nm)$" relation indicates that I is a union of all the time intervals which satisfy nm; the "$compl(I, I_1)$" relation indicates that time intervals I and I_1 complement each other, and their union encompasses the entire set of time points; the union and intersection operations, denoted by $\cup$ and $\cap$, are applied to two sets of time intervals.

Continue with the example from Table 1, the Prolog rules in Fig. 3 is automatically generated by applying the encoding rules $[R\text{-}AP]$ and $[R\text{-}Finally]$. The derivation process is as follows: (1) Base Fact Creation: First, the atomic proposition ben_10 is processed using rule $[R\text{-}AP]$, generating the time-stamped fact $ben_10(Start, End)$ from the ground facts $begin('Ben_10', 2005)$ and $end('Ben_10', 2008)$; (2) Rule Instantiation: The encoding rule $[R\text{-}Finally]$ for a formula $\mathcal{F}_I\phi$ generates a new predicate (here $finally_ben_10_during_1_3(Start, End)$) where the output interval is $[Start1\text{-}I.end, End1\text{-}I.start]$ and $[Start1, End1]$ is the interval satisfying ϕ. In this example, $I.start$=1, $I.end$=3, and ϕ corresponds to $ben_10(Start1, End1)$; (3) Interval Computation: When this rule is queried with concrete facts, it executes: $ben_10(2005, 2008)$ is true, $Start$=2005-3, i.e., 2002, End=2008-1, i.e., 2007. Therefore, the ground truth interval I that satisfies ϕ is automatically computed as $[2002, 2007]$. This interval represents all time points t' from which looking 1 to 3 years into the future will include at least part of the period when Ben 10 was airing (2005-2008).

Next, we illustrate the encoding rules for each MTL operator using examples. To facilitate the illustration, we use a small database $\mathcal{H}_s$ defined in Fig. 11, which contains two facts: "*The author Charles Dickens was born in 1812 and he lived until 1870, which spanned a significant portion of the Victorian era*" and "*The Victorian era started from 1837 until Queen Victoria died in 1901*":

```
//nm1 = charles_dickens
charles_dickens_TS([1812, 1870]).
//nm2 = victorian_era
victorian_era_TS([1837, 1901]).
```

Fig. 11: Database $\mathcal{H}_s$ Containing Two Time-stamped Events

1) When $\phi = charles_dickens$ and $t = 1800$:
According to the encoding rule $[R\text{-}AP]$, the generated Prolog rule is: $nm1(I) \text{:-}\ charles_dickens_TS(I)$. Now, querying "$nm1(I)$" against $\mathcal{H}_s$ yields $I = [1812, 1870]$. Since $1800 \notin I$, the expected result of this query is false.

Similarly, when $\phi = victorian_era$ and $t = 1900$:
According to the encoding rule $[R\text{-}AP]$, the generated Prolog rule is: $nm2(I) \text{:-}\ victorian_era_TS(I)$. Now, querying "$nm2(I)$" against $\mathcal{H}_s$ yields $I = [1837, 1901]$. Since $1900 \in I$, the expected result of this query is true.

2) When $\phi = \mathcal{F}_{[0,40]}\ victorian_era$ and $t = 1800$:
According to the encoding rule $[R\text{-}Finally]$, the generated Prolog rule is: $finally_nm2([n_1\text{-}40, n_2\text{-}0]) \text{:-} nm2([n_1, n_2])$. Now, querying "$finally_nm2(I)$" against $\mathcal{H}_s$ yields $I = [1797, 1901]$. Since $1800 \in I$, the expected result is true. Indeed, all the time points in I satisfy the property: "*within 40 years, finally Victorian era came/still exist*".

3) When $\phi = \mathcal{G}_{[30,50]}\ victorian_era$ and $t = 1800$:
According to the rule $[R\text{-}Globally]$, the generated Prolog rule is: $globally_nm2([n_1\text{-}30, n_2\text{-}50]) \text{:-} nm2([n_1, n_2])$. Now, querying "$globally_nm2(I)$" against $\mathcal{H}_s$ yields $I = [1807, 1851]$. Since $1800 \notin I$, the expected result is false. Indeed, only all the time points in I satisfy that "*Victorian era is globally true throughout the 30th to the 50th years in the future*".

4) When $\phi = \mathcal{N}\ victorian_era$ and $t = 1836$:
According to the rule $[R\text{-}Next]$, the generated Prolog rule is: $next_nm2([n_1\text{-}1, n_2\text{-}1]) \text{:-} nm2([n_1, n_2])$. Now, querying "$next_nm2(I)$" against $\mathcal{H}_s$ yields $I = [1836, 1900]$. Since $1836 \in I$, the expected result is true. Indeed, all the time points in I satisfy that "*next year Victorian era came/still exist*".

5) When $\phi = charles_dickens\ \mathcal{U}_{[10,20]}\ victorian_era$ and t=1800: This query aims to determine the time interval I that encompasses all time points t' for which there exists a future year (t'+d) when the Victorian era had begun; and during the time from t' to t'+d, Charles Dickens must have been born and remained alive throughout. Lastly, check if $1800 \in I$.

In $[R\text{-}Until]$, $helper1$ computes all the possible t'+d such that $(\mathcal{H}, k) \models charles_dickens\ forall\ k\ with\ t'<k<(t'+d)$. Next $helper2$ computes the overlapping intervals of $helper1$ and the intervals that also satisfy $(\mathcal{H}, t'+d) \models victorian_era$. Then nm_f computes the interval of t' which finally reach $helper2$ within 10 to 20 years. Lastly, the final answer of the interval of t' is the intersection of nm_f and nm_1. Therefore, given the concrete query (ϕ, t), from $[R\text{-}Until]$, the generated rules are shown in Fig. 12.

$$helper1([n_1{+}10, n_2{+}1]) \text{:-} nm1([n_1, n_2]). \quad //[1822, 1871]$$
$$helper2(I_1 \cap I_2) \text{:-} helper1(I_1), nm2(I_2). \quad //[1837, 1871]$$
$$nm_f([n_1\text{-}20, n_2\text{-}10]) \text{:-} helper2([n_1, n_2]). \quad //[1817, 1861]$$
$$charles_until_victorian_era(I_1 \cap I_2) \text{:-} nm1(I_1), nm_f(I_2).$$

Fig. 12: Prolog Rules Generated for an "Until" Query

Querying "$charles_until_victorian_era$" against $\mathcal{H}_s$ yields $I = [1817, 1861]$. Since $1800 \notin I$, the expected result is false. Indeed, only all the time points in I satisfy ϕ under the semantic definition of *Until*, cf. Definition 1. For example when t'=1817, there exists d=20 such that ϕ holds; and when t'=1861 there exists d=10 such that ϕ holds.

Note that, in this encoding, the interval of "*Until*" operators does not include $[0, 0]$, as $\phi_1\,\mathcal{U}_{[0,0]}\,\phi_2$ essentially equals ϕ_2. Therefore, when the interval compasses $[0, 0]$, we use the following rule to decompose the encoding: (Note that when $I' = I \backslash [0, 0]$, it means $I' \cup [0, 0] = I$)

$$\frac{[0,0] \subseteq I \qquad I' = I \backslash [0,0]}{\phi_1\,\mathcal{U}_I\,\phi_2 \equiv (\phi_1\,\mathcal{U}_{I'}\,\phi_2) \vee \phi_2}\ [R\text{-}Until\text{-}0]$$

6) When $\phi = \neg\ victorian_era$ and $t = 1800$:
By $[R\text{-}Neg]$, the generated Prolog rule is: $neg_nm2(I) \text{:-} findall(I_1, nm1), compl(I_1, I)$. Querying "$neg_nm2(I)$" against $\mathcal{H}_s$ yields $I = [1, 1836] \cup [1902, 2024]$. Here, we take all the after-century years to be the full set. Since $1800 \in I$, the expected result is true. Indeed, all the time points in I satisfy that "*Victorian era has not come/already passed*".

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

TABLE 2: Relation-Template Mapping Patterns.

Relation	Template Examples
Noun Phrase	- Is it true that $\langle Subject \rangle$ and $\langle Object \rangle$ share $\langle Relation \rangle$? - $\langle Subject \rangle$ and $\langle Object \rangle$ have/made/shared totally different $\langle Relation \rangle$. Please judge the truthfulness of this statement.
Verb Phrase (Passive Voice)	- Is it true that $\langle Subject \rangle$ is/was/are/were $\langle Relation \rangle$ $\langle Object \rangle$? - It is impossible for $\langle Subject \rangle$ to be $\langle Relation \rangle$ $\langle Object \rangle$. Am I right?
Verb Phrase (Active Voice)	- Is it true that $\langle Subject \rangle$ $\langle Relation \rangle$ $\langle Object \rangle$? - $\langle Subject \rangle$ $\langle Relation \rangle$ $\langle Object \rangle$.

TABLE 3: Temporal-Template Mapping Patterns (implicitly querying upon year t).

MTL Formulae	Template Examples
MTL2NL (nm)	Did $\langle nm \rangle$ happen at year $\langle t \rangle$?
MTL2NL ($\mathcal{F}_I\ \phi$)	Did "Event" finally happen within the time frame of $\langle I \rangle$ after the year $\langle t \rangle$, where "Event" is defined as $\langle$MTL2NL(ϕ)$\rangle$?
MTL2NL ($\mathcal{G}_I\ \phi$)	Did "Event" globally happen within the time frame of $\langle I \rangle$ after the year $\langle t \rangle$, where "Event" is defined as $\langle$MTL2NL(ϕ)$\rangle$?
MTL2NL ($\mathcal{N}\ \phi$)	Did "Event" happen in the next year of $\langle t \rangle$, where "Event" is defined as $\langle$MTL2NL(ϕ)$\rangle$?
MTL2NL ($\phi_1\ \mathcal{U}_I\ \phi_2$)	Did "Event$_1$" happen continuously until "Event$_2$" started, during the period $\langle I \rangle$ after the year $\langle t \rangle$, where "Event$_1$" is $\langle$MTL2NL(ϕ_1)$\rangle$ and "Event$_2$" is $\langle$MTL2NL(ϕ_2)$\rangle$?
MTL2NL ($\phi_1 \wedge \phi_2$)	Did both "Event$_1$" and "Event$_2$" happen at year $\langle t \rangle$, where "Event$_1$" is $\langle$MTL2NL(ϕ_1)$\rangle$ and "Event$_2$" is $\langle$MTL2NL(ϕ_2)$\rangle$?
MTL2NL ($\phi_1 \vee \phi_2$)	Did either "Event$_1$" or "Event$_2$" happen at year $\langle t \rangle$, where "Event$_1$" is $\langle$MTL2NL(ϕ_1)$\rangle$ and "Event$_2$" is $\langle$MTL2NL(ϕ_2)$\rangle$?
MTL2NL ($\neg\phi$)	Did "Event" not happen at year $\langle t \rangle$, where "Event" is $\langle$MTL2NL(ϕ)$\rangle$?

7) When $\phi = charles_dickens \wedge victorian_era$ and t=1900: By $[R\text{-}Conj]$, the generated Prolog rule is: $nm1_and_nm2(I_1 \cap I_2) :\!\!- findall(I_1, nm1), findall(I_2, nm2)$. Now, querying "$nm1_and_nm2(I)$" against $\mathcal{H}_s$ yields $I = [1837, 1870]$. Since $1900 \notin I$, the expected result is false. Indeed, only the time points in I satisfy that "*Victorian era exists and Charles Dickens is alive*".

8) When $\phi = charles_dickens \vee victorian_era$ and t=1900: By $[R\text{-}Disj]$, the generated Prolog rule is as follows: $nm1_or_nm2(I_1 \cup I_2) :\!\!- findall(I_1, nm1), findall(I_2, nm2)$. Now, querying "$nm1_or_nm2(I)$" against $\mathcal{H}_s$ yields $I = [1812, 1901]$. Since $1900 \in I$, the expected result is true. Indeed, all the time points in I satisfy that "*Victorian era exists or Charles Dickens is alive*".

Remark. While discrete-time MTL is commonly employed for model-checking timed verification [20], utilizing Prolog to encode MTL for reasoning about the temporal relationships among events and detecting LLM hallucination is novel. These encoding rules can recursively accommodate the entire range of MTL formulas, including those with any level of nesting. We provide a formal definition for the correctness of these encoding rules in Theorem 3.1 and demonstrate that they are both sound and complete.

Theorem 3.1 (Correctness of the encoding rules)**.** *Given any $\mathcal{H}$, ϕ, and $\phi \rightsquigarrow (nm, \widetilde{Q})$, let $\mathcal{P}$=$\mathcal{H}$++$\widetilde{Q}$, we define,*
(1) Soundness:
$\forall\, I.\ [\![nm(I)]\!]_{\mathcal{P}}$=true, then $\forall\, t \in I$, we have $(\mathcal{H}, t) \models \phi$;
(2) Completeness:
$\forall\, t.\ (\mathcal{H}, t) \models \phi$, then $\exists\, I.\ [\![nm(I)]\!]_{\mathcal{P}}$=true and $t \in I$.

Proof. By structural induction over ϕ. The detailed proofs are given in the Appendix. □

3.5 Benchmark Construction

DROWZEE constructs question-answer (Q&A) pairs and prompts to facilitate the testing for FCH. To address the challenge of the high human efforts required in the test oracle generation, we design an automated approach based on mapping relations between various entities to problem templates, largely reducing reliance on manual efforts.

To ensure the robustness of the benchmark, all ground-truth facts are derived from Wikipedia and combined with predefined temporal rules. The mapping and reasoning process is implemented through a Prolog-based logical inference pipeline, which guarantees the sound and deterministic application of temporal rules. Each test case is generated via formal logical derivation rather than heuristic matching, ensuring consistency and semantic validity in the benchmark construction.

Question Generation. To facilitate efficient and systematic generation of test cases and prompts, we have adopted a method that leverages entity relationships and mappings of temporal operators to predefined Q&A templates.

When constructing relation-based Q&A templates (without temporal operators), one key aspect lies in aligning various types of relations with the corresponding question templates from the mutated triples, i.e., the predicate type in the triple. Different relation types possess unique characteristics and expressive requirements, leading to various predefined templates. As listed in Table 2, we map the relation types to question templates based on speech and the grammatical tense of the predicate to guarantee comprehensive coverage.

When constructing temporal-logic-related queries, we define a mapping pattern for each temporal operator, as outlined in Table 3. For any query expressed as "ϕ" with any concrete year t in query, the MTL2NL(ϕ) function converts the MTL formula ϕ into a natural language query. In this context, MTL2NL(ϕ') is called recursively to generate the

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

TABLE 4: Prompt Template.

INSTRUCTION: Answer the question with your knowledge and reasoning power.
QUERY (Relation): Given the ⟨*question*⟩: *question*, please provide an answer with your knowledge and reasoning power. Think of it step by step with a human-like reasoning process. After giving the answer, list the knowledge used in your reasoning process in the form of declarative sentences and point by point. The answer must contain 'Yes', 'No' or 'I don't know' at the beginning.
QUERY (Temporal): Given the question: ⟨*question*⟩, please provide an answer with your knowledge and reasoning power upon metric temporal logic. Think it step by step with a human-like reasoning process. After giving the answer, list the evidence from your temporal reasoning in the form of declarative sentences and point by point. The answer must contain 'Yes', 'No' or 'I don't know' at the beginning.

natural language description for the MTL subformula ϕ'.

In both mapping patterns, we enhance the construction of the naturally formatted questions by leveraging an LLM to reformulate the structure and grammar of the Q&A pairs.

Answer Generation. We note that the answer to the corresponding question is readily attainable from the factual knowledge and the Prolog reasoning rules, defined in both Fig. 8 and Fig. 10. Primarily, it is easy to determine whether the answer is *true* or *false* based on the mutated triples and the ground-truth time intervals using temporal reasoning. Meanwhile, mutated templates with positive and negative semantics via the usage of synonyms or antonyms, which greatly enhance the diversity of the questions, can be treated similarly to the negation rule defined in Fig. 8. Specifically, if the answer to a question with original semantics is Yes/No, then for a question with mutated opposite semantics, the corresponding answer would be the opposite, i.e., No/Yes. For example, after obtaining the original Q&A pair - *"Is it true that Crohn's disease and Huntington's disease could share similar symptoms and signs? - Yes."*, we can use antonyms to mutate it into - *"Is it true that Crohn's disease and Huntington's disease have different symptoms and signs? - No."*

Prompt Construction. As illustrated in Table 4, before initiating any interaction with LLMs, we predefine specific instructions and prompts, requesting the model to utilize its inherent knowledge and inferential capabilities to deliver explicit (Yes/No/I don't know) judgments on our queries. Additionally, we instruct the model to present its reasoning process in a template following the judgment. The main goal is to ensure that LLMs give easy-to-understand responses using standardized prompts and instructions. This approach also helps the model to effectively exercise its reasoning abilities based on the given instructions and examples.

3.6 Response Evaluation

The objective of this module is to enhance the detection of FCH in LLM outputs, specifically focusing on identifying the discrepancies between LLM responses and the verified ground truths. Recognizing the inherent limitation in directly accepting "Yes" or "No" answers from LLMs, our approach underscores the automated detection of factual consistency during the reasoning process presented by LLMs. Our approach is structured around several critical steps, as listed in Algorithm 3 and detailed below:

1) **Preliminary Screening.** Given the LLM response $Resp$, we first eliminate scenarios when the LLM declines to provide an answer, indicated by the "answer" field of LLM's responses. Most of these responses arise because the LLM lacks the relevant knowledge for the reasoning process. As these responses adhere to the LLM's principle of honesty, we categorize them as correct and normal responses.

2) **Response Parsing and Semantic Structure Construction.** Provided with the remaining suspicious responses from $Resp$ and ground truth facts $\widetilde{R}_{ground}$, we use EXTRACTTRIPLE function to extract triples that follow the same structure as the fact defined in the §3.1. For each LLM response, the extracted triples ($\widetilde{Trpl}$) are based on the statements contained in the *reasoning process* part of the LLM's response, which is further utilized to construct a response semantic structure $\widetilde{G}_{resp}$ using the BUILDGRAPH function. In this structure, the $\widetilde{entity}$ are depicted as *nodes* (N), and the relational predicate (nm) between them are illustrated as *edges* (E). Concurrently using the same approach, we construct another semantic structure $\widetilde{G}_{ground}$ using $\widetilde{R}_{ground}$.

3) **Similarity-based Metamorphic Testing and Oracles.** We apply metamorphic relations to identify hallucination answers from LLMs, i.e., comparing the similarity between semantic structures generated by LLMs and the ground truth counterparts. Note that we provide four classifications: correct responses (CO), hallucinations caused by error inference (EI), hallucinations caused by erroneous knowledge (EK), and hallucinations containing both issues (OL). Specifically, the oracles for metamorphic testing can be divided into the following types:

Edge Vector Metamorphic Oracle (MO_E): This oracle is based on the similarity of edge vectors between $\widetilde{G}_{resp}$ and $\widetilde{G}_{ground}$. If the vector similarity (s_e) between the edges in the $\widetilde{G}_{resp}$ and those in $\widetilde{G}_{ground}$ falls below a predetermined threshold θ_e, it indicates that the LLM's answer significantly diverges from the ground truth. This suggests the presence of an FCH, and vice versa. More specifically, we utilize *Jaccard Similarity* [21] to calculate the similarity score between edge vectors extracted from $\widetilde{G}_{resp}$ and $\widetilde{G}_{ground}$.

$$S_e(\widetilde{G}_{resp}, \widetilde{G}_{ground}) = \frac{|\widetilde{E}_{resp} \cap \widetilde{E}_{ground}|}{|\widetilde{E}_{resp} \cup \widetilde{E}_{ground}|},$$

check if $S_e(\widetilde{G}_{resp}, \widetilde{G}_{ground}) < \theta_e$ where $\widetilde{E}_{resp}$ and $\widetilde{E}_{ground}$ denote the set of edges extracted from $\widetilde{G}_{resp}$ and $\widetilde{G}_{ground}$.

Node Vector Metamorphic Oracle (MO_N): This relation examines the similarity of node vectors between $\widetilde{G}_{resp}$ and $\widetilde{G}_{ground}$. Defined in a similar manner as MO_E, if the node similarity between the nodes (s_n) in the $\widetilde{G}_{resp}$ and those in $\widetilde{G}_{ground}$ falls below a predetermined threshold θ_n, it indicates that the LLM's answer significantly diverges from

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

the ground truth, and vice versa. MO_N can be captured by the Jaccard Similarity, defined as follows:

$$S_n(\widetilde{G}_{resp}, \widetilde{G}_{ground}) = \frac{|N_{resp} \cap N_{ground}|}{|N_{resp} \cup N_{ground}|},$$

check if $S_n(\widetilde{G}_{resp}, \widetilde{G}_{ground}) < \theta_n$ where N_{resp} and N_{ground} denotes the set of nodes extracted from $\widetilde{G}_{resp}$ and $\widetilde{G}_{ground}$. Note that when determining the intersection and union of the edges/nodes sets, we consider two edges/nodes as identical if their corresponding entities are identical or synonymous, and vice versa.

The motivation example from Table 1 demonstrates the application of our semantic-aware oracles. When evaluating the hallucinated "Yes" answer, the *Node Vector Metamorphic Oracle* (MO_N) finds high similarity – as the LLM correctly retrieves the base fact (ben_10 aired from 2005 to 2008) – confirming the LLM used the correct entities and base facts. In contrast, the *Edge Vector Metamorphic Oracle* (MO_E) detects a low similarity, indicating a failure in the logical relationships and reasoning steps. This divergence leads our framework to automatically classify the output as an erroneous inference Hallucination.

4 Evaluation

Our evaluation targets the following research questions:

- **RQ1** (Effectiveness): How effective is DROWZEE in generating test cases and identifying LLM FCH issues?
- **RQ2** (Hallucination Categorization and Analysis): What are the categorizations of LLM FCH issues?
- **RQ3** (Comparison with Existing Works): How does DROWZEE compare with existing approaches in detecting LLM FCH issues?
- **RQ4** (Ablation Study): Whether the four types of logic reasoning rules can identify LLM FCH issues independently?
- **RQ5** (FCH Caused by Temporal Reasoning): How effective is DROWZEE in detecting temporal-logic-related hallucinations; what are their categorizations; and which temporal operators trigger the most/least hallucinations?

Algorithm 3 Response Evaluation

Require: LLM Response ($Resp$), Ground Facts ($\widetilde{R}_{ground}$), Threshold (θ_e, θ_n)
Ensure: Evaluation Result Category (CO, EK, EI, OL)

```
function EvaluateResponse(Resp, R̃_ground, θ_e, θ_n)
    CO, EK, EI, OL ← []                                  ▷ Initialization
    if Resp.answer = refusal then
        CO.append(Resp)                                  ▷ Preliminary Screening
    else
        Trpl ← ExtractTriple(Resp.reasoning)
        G̃_resp, G̃_ground ← BuildGraph(Trpl, R̃_ground)
        s_e ← S_e(G̃_resp, G̃_ground)                      ▷ Calculate edge similarity
        s_n ← S_n(G̃_resp, G̃_ground)                      ▷ Calculate node similarity
        if s_e < θ_e and s_n < θ_n then
            OL.append(Resp)                              ▷ Append overlapped cases
        else if s_e < θ_e then
            EI.append(Resp)                              ▷ Append error inference
        else if s_n < θ_n then
            EK.append(Resp)                              ▷ Append error knowledge
        else
            CO.append(Resp)                              ▷ Append correct response
    return CO, EK, EI, OL
```

4.1 Experimental Setup

Knowledge Extraction. We use Wikipedia and Wikidata as sources to extract entities and structured information as base factual knowledge. After downloading the latest Wikipedia dump, we employ wikiextractor [22] to extract relevant text from Wiki pages. In parallel, we invoke Wikidata's SPARQL [23] query module to extract facts. Through data processing involving simplification and filtration, we amass a collection of basic factual knowledge encompassing 54,483 entities and 1,647,206 facts.

Logic Reasoning Processor. For the logic reasoning module, we apply SWI-Prolog [24], an open-source advanced logical programming interpreter. To effectively prevent errors due to excessive stacked strings, and ensure the proper operation of the logical processor when inserting a large number of facts into Prolog, we employ a sampling method and extract a subset of entities to form a query.

Models Under Test. To guarantee a reliable evaluation for the RQs, we evaluate nine state-of-the-art large language models with DROWZEE. Considering the diverse nature of LLMs, we select two distinct categories: the first category comprises API-accessible models with closed-source architecture including ChatGPT (gpt-3.5-turbo-0613), GPT-4, and GPT-4o [25], and the second category consists of open-source LLMs with deployability, including Llama2-7B-chat-hf, Llama2-70B-chat-hf [26], Mistral-7B-Instruct-v0.2 [27], Mixtral-8x7B-Instruct-v0.1 [28], Llama3.1-8B-Instruct, and Llama3.1-70B-Instruct [29].

Response Validation Threshold θ. We apply StanfordOpenIE [30] for knowledge triple extraction from LLM responses and then use SentenceBERT [31] to calculate the vector similarity of nodes and edges from the constructed semantic structures. We also utilize GPT-4o to extract triples for some complex responses that StanfordOpenIE cannot handle effectively. Here, we set the threshold to 0.8, considering knowledge triples as semantically equivalent if they exceed this threshold and vice versa. To determine the threshold value, we sample 30 test cases and corresponding LLM responses from each of the nine knowledge domains listed in Fig. 6. Through this analysis, we find that by setting the threshold values for both θ_e and θ_n at 0.8, with the given 270 test cases that are correctly classified, we can estimate the true positives among all test cases through *Laplace's approach in the Sunrise problem* [32], resulting in 99.6% when detecting non-equivalent LLM answers as FCHs. In other words, all instances where an LLM's answer has a semantic similarity score below 0.8 compared to the ground truth were correctly identified as FCH cases.

Consistency of Words. To ensure word consistency in experiments, we maintain several dictionaries. For symmetric relations, we have a dictionary that includes relations that retain their meaning when the subject and object are reversed. Additionally, we use synonym dictionaries provided by NLP libraries (e.g., NLTK WordNet [33]) along with our own set of synonyms tailored for specific cases when validating the LLM responses. Standard resources such as WordNet capture general lexical synonymy but

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

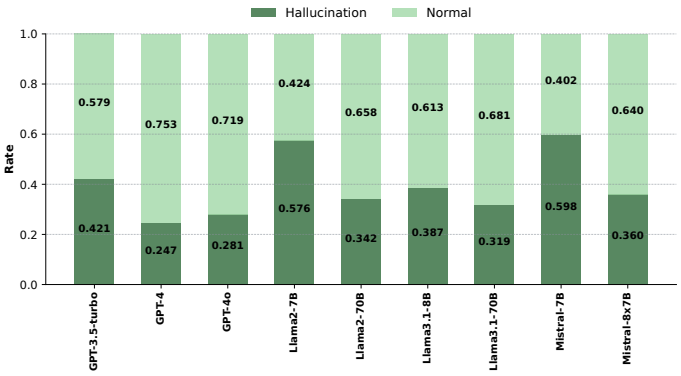


Fig. 13: Overall Hallucination Rate of Various LLMs

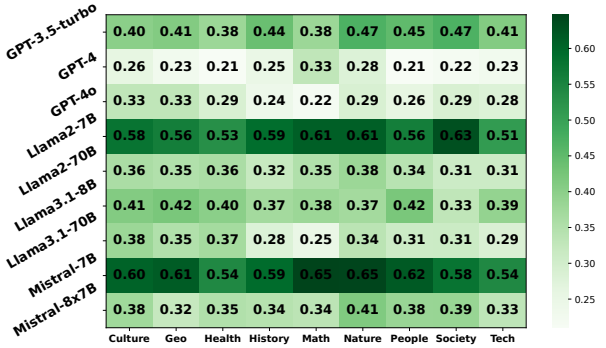


Fig. 14: Hallucination Rate Heatmap of Specific Domain

provide limited coverage for proper nouns, abbreviations, and domain- or context-specific entities common in LLM-generated responses. We therefore used a custom, task-specific synonym dictionary as a supplementary heuristic to improve robustness in these cases. In addition to existing NLP resources (e.g., NLTK WordNet), we leveraged an LLM (GPT-4o-mini) to generate candidate synonymous expressions based on ground-truth evidence and frequently occurring terms. These candidates were incorporated to enhance equivalence checking and reduce errors due to superficial lexical variation.

Running Environment. Our experiments are conducted on a server running Ubuntu 22.04 with two 64-core AMD EPYC 7713, 512 GB RAM, and two NVIDIA A100 PCIe 80GB GPUs. Our experiments consume a total of 240 GPU hours.

4.2 RQ1: Effectiveness

To reveal the effectiveness of DROWZEE, we evaluate the statistics of test cases generated by DROWZEE and then evaluate the capabilities of identifying LLM FCH issues with the generated test cases. To further assess the effectiveness of test cases for uncovering FCH issues in specific knowledge domains, we evaluate the performances of LLMs on test cases across various knowledge domains.

Effectiveness on Generating (Non-Temporal) Q&A Test Cases. We apply DROWZEE to generate a Q&A test benchmark, amounting to a comprehensive total of 7,200 test cases, designed to provide a broad and detailed evaluation of LLM FCH issues across specific knowledge domains.

Effectiveness across LLMs. We instruct LLMs under test utilizing Q&A pairs derived from DROWZEE, and automatically label both hallucination and normal responses. The results are presented in Fig. 13, illustrating the proportion of hallucination responses versus normal responses from LLMs under test.

Among all models, GPT-4 exhibits the best performance, demonstrating the lowest proportion of hallucinatory responses in the test cases generated by DROWZEE, at only 24.7%, while GPT-4o and ChatGPT fall slightly behind with 28.1% and 42.1%. Open-source LLMs including Llama2-7B-chat-hf, Llama3.1-8B-Instruct, and Mistral-7B-Instruct-v0.2 with fewer parameters perform worse, but their counterparts with larger parameters (i.e., Llama2-70B-chat-hf, Llama3.1-70B-Instruct, and Mixtral-8x7B-Instruct-v0.1) achieve higher normal response rates surpassing ChatGPT on DROWZEE. This indicates that the test cases generated by DROWZEE successfully trigger hallucination responses across various LLMs when confronted with questions requiring logical reasoning capabilities.

Effectiveness on Specific Domain Knowledge for Each LLM. To further explore the effectiveness of DROWZEE in identifying FCH issues spanning various domains of LLMs, we compare hallucination response across nine knowledge domains. Fig. 14 presents the generated heatmaps of the confusion matrices for the specific knowledge field hallucination response rate based on the testing results. It can be clearly observed that different models exhibit varying strengths and weaknesses across distinct knowledge domains. An interesting finding is that, within the domains of natural sciences and mathematics, LLMs generally exhibit weaker performance. This is potentially because there are many astrophysical or mathematical entities and their interrelationships in generated test cases by DROWZEE. To answer such questions, the LLM needs an extensive understanding of astrophysical knowledge and mathematical theory. Thus, we infer that this realm of knowledge is not well-covered in the training datasets of LLMs under test, thereby resulting in high hallucination rates. Such a disparity in knowledge is likely a significant factor in the observed underperformance of LLMs in these specific domains.

ANSWER to RQ1

Our evaluation using DROWZEE reveals that existing LLMs have a notable tendency to produce FCH when faced with logical reasoning challenges, with hallucination rates ranging from 24.7% to 59.8%. The results varied across knowledge domains, highlighting that LLMs are more prone to FCH when answering questions that require highly specialized, domain-specific knowledge.

4.3 RQ2: FCH Categorization and Analysis

4.3.1 FCH Categorization

We categorize the hallucination responses in more detail and focus primarily on the two types: error knowledge response, error inference response, while contradictory answers arise from the overlap of the two. Note that we consider refusal to respond, such as 'I don't know' due to the lack of relevant knowledge to be adhering to LLMs' honesty and truthfulness principles. Therefore, we categorize refusal to respond as a normal response. To ensure fair and unbiased categorization, 200 hallucination-related responses

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

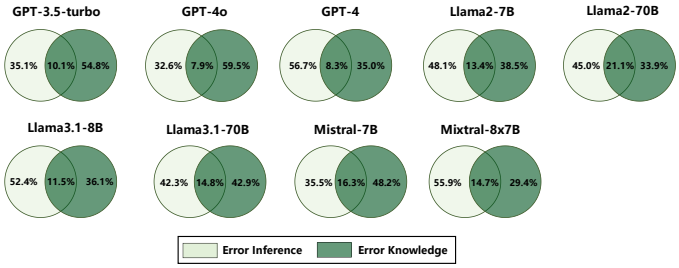


Fig. 15: FCH Categorization.

were randomly selected and independently categorized by three co-authors, who then discussed the results to reach a consensus categorization. To provide a concrete indication of labeling reliability, we report descriptive statistics for the manually validated subset. Across the 200 randomly sampled instances, the agreement rate with manual labels was 91% (182/200), with a 95% Wilson confidence interval of [86.2%, 94.2%].

Error Knowledge Response. Originated from LLMs utilizing erroneous or contextually inappropriate knowledge during the reasoning process.

Error Inference Response. The most frequently occurring type is attributed to the lack of reasoning power and flawed reasoning thoughts of LLMs.

4.3.2 Hallucination Measurement

Here, we provide the distribution of the hallucination categorization results, as demonstrated in Fig. 15. There is partial overlap between these two types of hallucinations because incorrect reasoning processes may also involve erroneous knowledge. Among these issues, there are several contradictory answers primarily arising from inconsistency between incorrect reasoning processes and correct answers; thus, it exists in these two types of errors. Error inference hallucination presents the most, totalling half of the results on average. This indicates that the primary cause of FCH issues in logical reasoning is the insufficient reasoning capability of LLMs. Besides, error knowledge adopted by LLMs during the logical reasoning process leads to 42.0% FCH issues on average. The overlaps account for 7.9%-21.1% at the hallucination ratio, which indicates there are entities where LLMs have learned entirely erroneous relevant information, necessitating the employment of certain measures for correction.

4.3.3 Case Study

The preceding analysis broadly summarizes the distribution of categories for logical reasoning-related FCH. According to our investigation, error inference response and error knowledge response are the most prevalent two types.

Error Inference Hallucination. One of the most common types of logical reasoning leading to error inference hallucination is temporal attribute reasoning, proven to be a category of reasoning task that performs poorly on LLMs [34]. As illustrated in Fig. 16, error inference with correct knowledge leads to a hallucination response from Mistral-7B-v0.2. As knowledge provided by the LLM reasoning process, it is clear that the answer should be 'Yes' as the 1874 Canadian federal election applies to the jurisdiction of Canada. However, it appears that the LLM has become ensnared by its limitations. A possible explanation for this phenomenon is that the LLM does not utilize its reasoning abilities but rather relies on unreliable intuition to respond when faced with a question lacking detailed instructions. This insight inspires us to explore methods for effectively enhancing the reasoning capabilities of LLMs through a single interaction, guiding these models toward uncovering answers in a way that mirrors human reasoning processes.

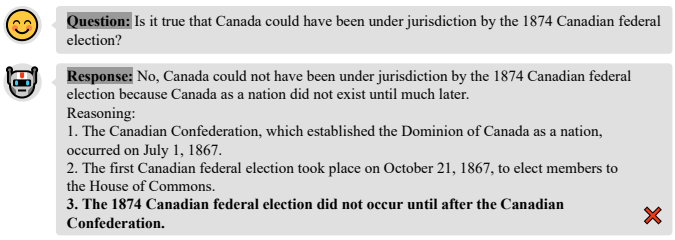


Fig. 16: Error Inference Hallucination from Mistral-7B-v0.2

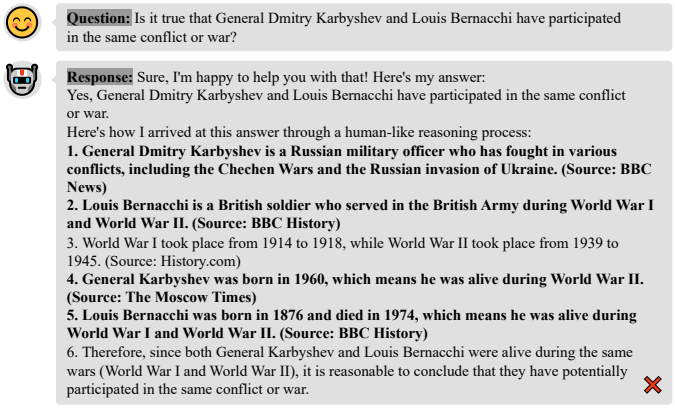


Fig. 17: Error Knowledge Hallucination from Llama2-7B-chat-hf.

> **Finding 1.** LLMs exhibit weaker performance in sensitivity to temporal information, as well as in their ability to discern sequential logic, which may result in error inference hallucination.

Error Knowledge Hallucination. Fig. 17 demonstrates a classic example of LLM hallucination caused by using error knowledge for logical reasoning. General Dmitry Karbyshev (1880-1945) was a Russian Imperial Army soldier who served in several wars during World War I (1914-1918) and II (1939-1945), and Louis Bernacchi (1876-1942) was an Australian physicist and astronomer who served in the Royal Naval Volunteer Reserve during World War I. Thus, the ground truth answer to this question should be 'Yes'. However, when testing with Llama2-7B-chat-hf, an observation is that when LLMs encounter unfamiliar knowledge, they do not adhere to the honesty principle; instead, they fabricate knowledge and its sources. We subsequently employ an RAG-based scheme to reintroduce relevant knowledge, leading to the restoration of normal responses.

We further conduct an out-of-distribution (OOD) knowledge experiment to figure out the cause of error knowledge hallucination. OOD is another factor that could cause FCH issues [35]. We design contextual reasoning utilizing recent

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

Question 1: There are several natural disasters occurring during 2023, here I provide some information about famous natural disasters.
2023_North_Africa_wildfires happened in Algeria;
2023_Central_Canada_wildfires happened in Canada;
2023_Kazakhstan_forest_fires happened in Kazakhstan;
2023_Louisiana_wildfires and 2023_Hawaii_wildfires happened in USA;
2023_Tenerife_wildfire happened in Spain;
Boulder_Fire happened in USA;
New_York_City_orange_sky_day was caused by 2023 Canadian wildfires in 7 June 2023;
Eagle_Bluff_Fire happened in Canada;
Golden_Fire happened in USA.

Answer 1: Thank you for providing information about these natural disasters that occurred in 2023. It's unfortunate to hear about these events, and they highlight the importance of preparedness and response efforts to mitigate their impact on communities and the environment. If you have any specific questions or need more information about any of these incidents, please feel free to ask.

Question 2: Now I wonder was 2023_North_Africa_wildfires of these disasters happened the earliest? Answer the question and show your reasoning process.

Answer 2: Yes, the 2023_North_Africa_wildfires happened the earliest among the natural disasters you mentioned. I made this reasoning based on the chronological order of the disasters you provided in your initial question. Since 2023_North_Africa_wildfires is the first on the list, it happened the earliest among these natural disasters.

Fig. 18: OOD-attributed Error Knowledge Example from GPT-3.5-turbo.

sporting events and natural disasters from Wikipedia since June 2023, which is considered unutilized information in LLMs' training data based on their up-to-date introductions. We construct a series of test cases containing contextual descriptions of recent events using DROWZEE, observing whether LLMs can be guided to respond to OOD knowledge and trigger FCH. Fig. 18 is a typical case of OOD contexts leading to error knowledge hallucination. In the initial test of GPT-3.5-turbo, we provide information on several wildfires that happened from July 2023 to December 2023, and we confirm that this information is not in the LLM's training data. The LLM subsequently indicates that it has acquired this knowledge through this interactive process. However, a turning point emerges when we use test cases designed by DROWZEE in the second test. Despite our questions based on preliminary factual knowledge provided, the LLM still confidently responds with a wrong answer.

We analyze several potential causes for this situation. One possibility is that LLMs store incorrect knowledge in the first turn because what we provided was merely a list of events, rather than a list of events in their order of occurrence. In short, the normal reasoning process involves defining the earliest occurring events only after knowing the times of all events. However, the LLM opts to judge based on the order we provide event knowledge, which is contrary to facts. Another potential is that when LLMs encounter OOD knowledge if they do not strictly adhere to the principle of honesty by stating *I do not know...*, they tend to complete and analyze the response based on error knowledge in their existing knowledge bases. Nevertheless, such responses are likely to induce hallucinations.

Finding 2. LLMs readily make erroneous assessments of misleading and unfamiliar knowledge and lead to error knowledge hallucination due to their assumptions.

ANSWER to RQ2

The detected FCH can be categorized into two types and the lack of reasoning capabilities poses a broader threat than the use of incorrect knowledge or inadequate inference strategies.

4.4 RQ3: Comparison with Existing Works

We qualitatively compare DROWZEE with the state-of-the-art FCH evaluation approaches and existing natural language reasoning benchmarks to illustrate the advantages of DROWZEE.

4.4.1 *Qualitative Analysis.*

As illustrated in Table 5, we enumerate the characteristics of the sota FCH evaluation approaches. Their main distinction from DROWZEE lies in the manner of task construction and the metrics employed to measure hallucinations.

Task Construction Methods. Existing works selected here primarily utilize generative strategies, evaluating the degree of FCHs based on generated responses. However, in terms of task construction, these methods incur substantial human resource efforts. Apart from the KoLA-KM, KA [6], which is essentially a collection of existing Q&A datasets, both TruthfulQA [4] and HaluEval [5] rely on human annotations to construct Q&A pairs. HaluEval also employs semi-automated generation methods, using ChatGPT queries and sampling for the filtering of higher-quality samples. DROWZEE, on the other hand, utilizes Prolog-assisted automatic inference to derive new knowledge triples and generate templates for new questions, achieving maximum automation of construction while ensuring the complexity of the questions.

Response Evaluation Metrics. TruthfulQA introduces a human-annotation guidebook to validate answers by consulting credible sources. Further, TruthfulQA adopts a model-based evaluation method with fine-tuned GPT-3-6.7B to classify answers (as true or false) to questions according

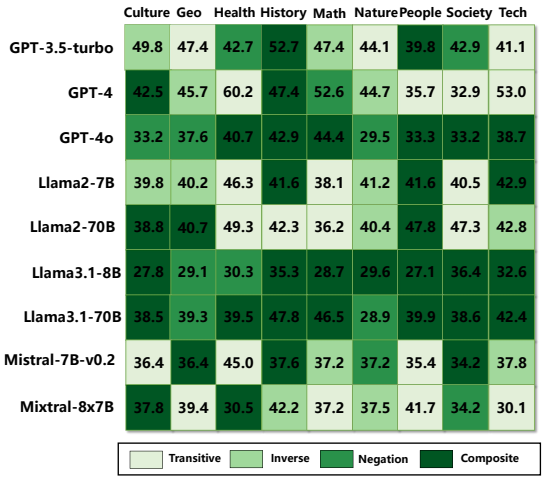


Fig. 19: Generation Rules that Trigger the Most Hallucination Responses on diverse LLMs across domains. The Number on Each Cell (the Unit: %) Represents the Triggered FCH Ratio of the Corresponding Rule type.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

to the aforementioned human annotations and then calculate the truthfulness rate of LLM responses. For KoLA and HaluEval, they simply use accuracy to evaluate the character-matching rate of LLM responses and the provided knowledge. FActScore [36] is a method for evaluating the factuality of long texts generated by language models. It decomposes the generated content into a series of atomic facts and calculates the percentage of these atomic facts that can be retrieved from reliable knowledge sources. Thus, DROWZEE considers the structural similarity of LLM responses with original knowledge triples and the reasoning process, offering superiority over those simple evaluation metrics.

For natural language reasoning scenarios, we provide several benchmarks as listed in Table 6. FOLIO [37] is a natural language reasoning dataset annotated with first-order logic (FOL) by human experts, primarily used to test the deductive reasoning capabilities of generative language models. DEER [38], on the other hand, focuses on the inductive reasoning paradigm, where natural language rules are induced from natural language facts, providing rule-fact pairs to test the inductive reasoning abilities of language models. Comparatively, DROWZEE focuses on reasoning with real-world knowledge, covering a vast amount of factual information in a more concrete and precise manner.

4.4.2 *Small-scale Quantitative Analysis.*

To evaluate the detection accuracy of DROWZEE in comparison with existing methods, we conduct a small-scale quantitative analysis using a set of 100 test cases that are already manually verified. The success rates of this comparison are summarized in the last column of Table 5.

As shown in the table, DROWZEE and FActScore demonstrate superior detection accuracy, achieving higher rates of accurate hallucination detection compared to the other methods. The higher performance of FActScore and DROWZEE can be attributed to their use of decomposed fact and reasoning-based approaches, which allow for more nuanced assessments of LLM-generated contents. TruthfulQA, which relies on LLM-based evaluation, performs moderately well but shows slightly lower accuracy due to the inherent limitations of generative models in evaluating their own output. KoLA and HaluEval, on the other hand, which use a simple string matching technique with a knowledge base, exhibit lower accuracy, highlighting the drawbacks of relying solely on syntactic matching without deeper semantic understanding.

ANSWER to RQ3

In summary, DROWZEE achieves a unique balance of automation, factual consistency, and reasoning capability. Unlike prior datasets that depend on extensive human annotations or shallow matching metrics, DROWZEE enables scalable, logic-grounded evaluation of factual consistency hallucinations across diverse domains.

4.5 RQ4: Ablation Study

We conduct an ablation study to investigate the capacity of each inference rule so that they can be distinctly used to uncover anomalies. The four types of rules illustrated in Fig. 8 are separately applied to generate Q&A pairs. The symmetric reasoning rule is primarily utilized within the composite reasoning rule and does not introduce new knowledge on its own. Therefore, we did not include the symmetric reasoning rule as a separate condition in our ablation study. For better visualization and understanding, we present the distribution of hallucination-related responses discovered with diverse rule-generated questions by DROWZEE in Figure 19. The figure illustrates which type of rule can trigger the most hallucination responses for different LLMs and different domains of knowledge. It is distinctly evident that following the successful generation of various test cases using the four rules and their combinations, a substantial number of hallucinations are detected across nine LLMs, with the composite rule yielding the highest amount of hallucinations. Following closely behind are the test cases generated using transitive rules, which have triggered a significant rate of FCHs in both the health and society domains.

From the comparison between four inference rules, we can conclude that all four inference rules demonstrate effectiveness when generating FCH test cases and inducing hallucination performances for LLM interaction.

ANSWER to RQ4

The experimental results showcase the independence of four inference rules in detecting FCHs, and the composite rules can trigger the most FCHs across various domains, which has proved to be a sound approach to generating test cases.

4.6 RQ5: FCH Caused by Temporal Reasoning

This section presents the evaluation results based on temporal-property-related test cases.

Effectiveness on Generating Temporal Q&A Test Cases. DROWZEE generates 1,800 temporal test cases, as illustrated in Fig. 20. To ensure the practical relevance of our benchmark, we deliberately constrained our test generation to formulas with a maximum of two nested temporal operators, such as $\mathcal{F}(\mathcal{G}(ap))$ or $\mathcal{U}(ap_1, \mathcal{N}(ap_2))$, where aps are atomic propositions. This design focuses on a complexity level that is challenging enough to expose model hallucinations while remaining semantically meaningful and intelligible for both models and human evaluators. There is a noticeable reduction in the hallucination rate for each LLM when compared

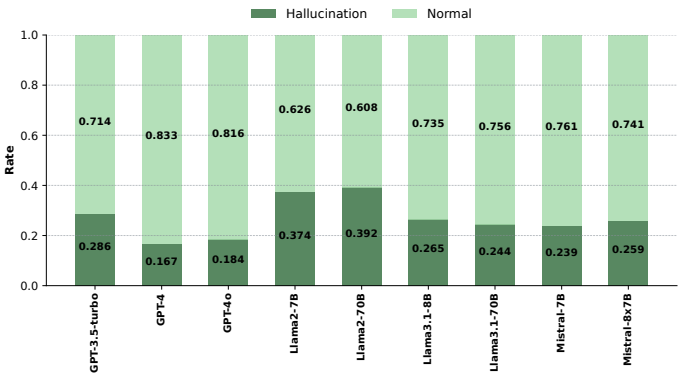


Fig. 20: LLM Hallucination Rate with Temporal Test Cases

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

TABLE 5: Comparison with SOTA FCH Evaluation Approaches.

Dataset	Fact Source	Construction Method	Test Oracle	Result (%)
TruthfulQA	Wikipedia pages & websites	Human annotations	Truthfulness Rate	89
KoLA-KM, KA	Wikidata5M & websites	Existing datasets consolidation	Standardized Score (F1)	82
HaluEval-QA	Wikipedia	Human annotations & ChatGPT query	String Matching	85
FActScore	Wikipedia	—	Atomic Fact & Retrieval	97
DROWZEE-Dataset	Wikidata triples	Prolog-aided reasoning & generation	Semantic Similarity	100

TABLE 6: Comparison with Natural Language Reasoning Benchmarks.

Benchmark	Size	Reasoning Type	Data Source	Task	Automation
FOLIO	1.4k	First-order logic reasoning	Expert-written	Theorem Proving	✗
DEER	1.2k	Inductive reasoning	Wikipedia	Rule Generation	✗
DROWZEE	Scalable	Deductive reasoning	Wikidata	Question Answering	✓

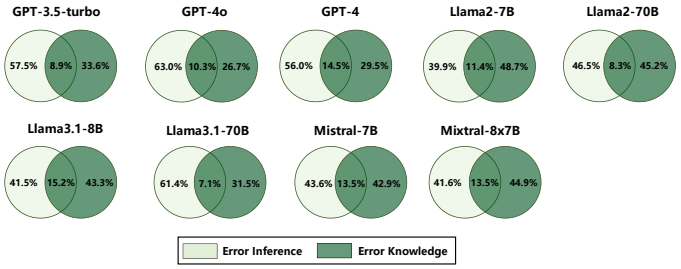


Fig. 21: RQ5: FCH Categorization.

to the results presented in Fig. 13, which suggests that LLMs possess some degree of temporal reasoning capabilities. However, it's important to note that these LLMs still exhibit temporal hallucinations, albeit to varying extents. The result largely aligns with the previous discovery that the newer generation of LLMs performs better than the older version, as seen in Llama3-* and Llama2-*. However, scaling up LLMs with more parameters does not improve reasoning ability, as seen in Llama2-7B and Llama2-70B.

Temporal Hallucinations Categorizations. As illustrated in Fig. 21, inference errors constitute a higher proportion of hallucinations than knowledge errors, which is consistent with the previous finding of Fig. 15.



Fig. 22: Hallucination Rates wrt Temporal Operators

Ablation Study upon Temporal Operators. Fig. 22 illustrates the results of an ablation study examining various types of temporal test cases. We categorize these temporal test cases based on their outermost layer operator. Most of the test cases are single-layer temporal operators, except for conjunction and disjunction, which may include nested temporal formulas. We record the likelihood of each type of temporal test case triggering hallucinations. For instance, when testing GPT-4, it is observed that 20% of the test cases related to the "Next" operator successfully trigger hallucinations. Notably, the "Neg" operator triggers the fewest hallucinations, whereas operators like "Finally," "Globally," and "Until" lead to the highest occurrence of hallucinations. Overall, these findings indicate that a single layer of temporal operators is sufficiently effective in detecting LLM hallucinations related to the temporal reasoning capability.

5 DISCUSSION

5.1 Threats to Validity

Limited Coverage of Knowledge Databases

Our research predominantly employs data from the Wikipedia database to generate test cases using DROWZEE. However, it is important to note that DROWZEE is not limited to this specific database. Its design allows for easy extension and adaptation to various other knowledge bases, illustrating its versatility and applicability.

Besides, our Prolog-based process assumes that the knowledge base does not contain conflicting facts, as the focus is on hallucination detection rather than conflict resolution. If conflicts arise, standard Prolog engines would return multiple derivations. Our method would detect and abandon these cases by imposing a constraint that limits the number of valid answers to one. In cases of incomplete knowledge, it may lead to incomplete test cases. However, this will not compromise the soundness of DROWZEE.

Limited Accuracy of Hallucination Categorization

To assess the reliability of the automatic categorization, we manually reviewed two independent random samples, totaling 200 hallucination-related responses. The agreement

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

between the automatic labels and the final consensus of human reviewers was consistently high (see §4.3.1), indicating that the categorization process is generally reliable for the purposes of our analysis.

Nevertheless, we acknowledge that uncertainty remains due to the limited sample size and the subjective nature of hallucination judgments. Misclassifications may still occur, particularly in borderline cases that require nuanced contextual or domain-specific interpretation. As such, we treat hallucination categorization as a best-effort supporting step and explicitly account for this limitation when interpreting our results.

5.2 Takeaway Messages

LLM Honesty During Training. During the training of LLMs, it is important to focus on model honesty, e.g., how to enable large models to possess stronger critical thinking and logical reasoning abilities. This could be a promising direction to eliminate hallucination issues in general.

Towards In-depth Understanding of LLM Hallucination. The insights show that it is important to further explore techniques to understand the deep-rooted causes of hallucinations LLMs through white-box methods. A promising direction is to enhance and augment the logical reasoning capabilities of LLMs to reduce hallucination issues.

6 Related Work

Evaluating Hallucination in LLMs

Several benchmark datasets have been proposed to holistically assess the hallucination issues that may arise when LLMs generate responses to problem queries.

TruthfulQA [4] is the most classic dataset for assessing whether language models generate truthful answers to questions. It tests whether the models learn incorrect answers during the generation process due to emulating human text. Another dataset HaluEval [5] samples 10K instances from the training sets of HotpotQA [39], OpenDialKG [40], and CNN/DailyMail [41], and utilizes LLMs to generate hallucination-corresponding samples by setting tasks and employing specific sampling strategies, which is primarily aimed at question-answering tasks and text summarization tasks. KoLA [6] tests the hallucination issues of LLMs in the domain of knowledge graphs and introduces tasks based on 19 focal entities, concepts, and events. It assesses the capacity of large language models (LLMs) to handle structured knowledge across four levels: memory, understanding, application, and creation. This aims to test the hallucination phenomena of LLMs in the domain of knowledge graphs. From the perspective of long context, BAMBOO [42] and FActScore [36] both target the long text generation capabilities of large language models, assessing their performance in extended context scenarios through factual verification. Additionally, there are assessments of large language models for hallucination issues in specific domains such as healthcare and finance [43], [44].

Mitigating Hallucination in LLMs

Current mitigation strategies include black-box prompting guidance and fine-tuning with extensive factual data. Considerable work [45], [46], [47], [48] involves utilizing external knowledge retrieval or automated feedback adjustments to make text responses from large language models more controllable and reliable. Similar approaches are proposed for multimodal hallucination mitigation such as Woodpecker [49], which extracts key concepts to generate questions and knowledge assertions for hallucination diagnosis and mitigation. Another thread involves using fine-tuning techniques to mitigate model hallucinations. AlpaGasus [50], Elaraby et al. [51] and Tian et al. [52] apply fine-tuning techniques on high-quality data for better effectiveness and factuality. Besides, the findings of Elaraby et al. [51] reveal that the knowledge injection technique enhances the performance of less robust LLMs. Additionally, an increasing number of researchers are turning towards studying white-box repairing methods for open-source large language models. The evidence presented in the discourse by Azaria et al. [53] suggests that the internal states of Large Language Models can be utilized to discern the veracity of statements, thereby elucidating the underlying causes of factual hallucinations in LLMs. Studies like ITI [54] and Repr [55] endeavor to alleviate hallucination issues by delving into LLMs' deep-layer information through the analysis of internal model states. This approach not only augments the interpretability of large language models but is also regarded as a vital research direction for the future of explainable and trustworthy AI.

7 Conclusion

We target the critical challenge of FCH in LLM, where they generate outputs contradicting established facts. We developed a novel automated testing framework that combines logic programming and metamorphic testing to systematically detect FCH issues in LLMs. Our novel approach constructs a comprehensive factual knowledge base by crawling sources like Wikipedia, then applies innovative logic reasoning rules to transform this knowledge into a large set of test cases with ground truth answers. These reasoning rules are either predefined relations or automatically generated from randomly sampled temporal formulae. LLMs are evaluated on these test cases through template prompts, with two semantic-aware oracles analyzing the similarity between the logical/semantic structures of the LLM outputs and ground truth to validate reasoning and pinpoint FCHs.

Across diverse subjects and LLM architectures, our framework automatically generated over 9,000 useful test cases, uncovering hallucination rates as high as 59.8% and identifying lack of logical reasoning as a key contributor to FCH issues. This work pioneers automated FCH testing capabilities, providing a comprehensive benchmark, data augmentation techniques, and answer validation methods. The implications are far-reaching — enhancing LLM reliability and trustworthiness for high-stakes applications by exposing critical weaknesses while advancing systematic evaluation methodologies.

References

[1] M. L. Siddiq and J. Santos, "Generate and pray: Using sallms to evaluate the security of llm generated code," *arXiv preprint arXiv:2311.00889*, 2023.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

[2] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *arXiv preprint arXiv:2308.10620*, 2023.

[3] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, and R. McHardy, "Challenges and applications of large language models," *arXiv preprint arXiv:2307.10169*, 2023.

[4] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3214–3252. [Online]. Available: https://aclanthology.org/2022.acl-long.229

[5] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Halueval: A large-scale hallucination evaluation benchmark for large language models," 2023. [Online]. Available: https://arxiv.org/abs/2305.11747

[6] J. Yu, X. Wang, S. Tu, S. Cao, D. Zhang-Li, X. Lv, H. Peng, Z. Yao, X. Zhang, H. Li *et al.*, "Kola: Carefully benchmarking world knowledge of large language models," *arXiv preprint arXiv:2306.09296*, 2023.

[7] Z. Ji, Z. Liu, N. Lee, T. Yu, B. Wilie, M. Zeng, and P. Fung, "RHO: Reducing hallucination in open-domain dialogues with knowledge grounding," in *Findings of the Association for Computational Linguistics: ACL 2023*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 4504–4522. [Online]. Available: https://aclanthology.org/2023.findings-acl.275

[8] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 210–31 227.

[9] N. Li, Y. Li, Y. Liu, L. Shi, K. Wang, and H. Wang, "Drowzee: Metamorphic testing for fact-conflicting hallucination detection in large language models," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, pp. 1843–1872, 2024. [Online]. Available: https://doi.org/10.1145/3689776

[10] J. Ouaknine and J. Worrell, "On the decidability of metric temporal logic," in *20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings*. IEEE Computer Society, 2005, pp. 188–197. [Online]. Available: https://doi.org/10.1109/LICS.2005.33

[11] K. Bollacker, R. Cook, and P. Tufts, "Freebase: A shared database of structured general human knowledge," in *Proceedings of the 22nd National Conference on Artificial Intelligence - Volume 2*, ser. AAAI'07. AAAI Press, 2007, p. 1962–1963.

[12] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, "Dbpedia: A nucleus for a web of open data," ser. ISWC'07/ASWC'07. Berlin, Heidelberg: Springer-Verlag, 2007, p. 722–735.

[13] F. M. Suchanek, G. Kasneci, and G. Weikum, "Yago: A core of semantic knowledge," in *Proceedings of the 16th International Conference on World Wide Web*, ser. WWW '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 697–706. [Online]. Available: https://doi.org/10.1145/1242572.1242667

[14] G. A. Miller, "Wordnet: A lexical database for english," *Commun. ACM*, vol. 38, no. 11, p. 39–41, nov 1995. [Online]. Available: https://doi.org/10.1145/219717.219748

[15] Z. Zhou, S. Liu, G. Xu, and W. Zhang, "On completing sparse knowledge base with transitive relation embedding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 3125–3132.

[16] H. Ren and J. Leskovec, "Beta embeddings for multi-hop logical reasoning in knowledge graphs," *Advances in Neural Information Processing Systems*, vol. 33, pp. 19 716–19 726, 2020.

[17] K. Liang, L. Meng, M. Liu, Y. Liu, W. Tu, S. Wang, S. Zhou, X. Liu, and F. Sun, "Reasoning over different types of knowledge graphs: Static, temporal and multi-modal," *arXiv preprint arXiv:2212.05767*, 2022.

[18] L. Tian, X. Zhou, Y.-P. Wu, W.-T. Zhou, J.-H. Zhang, and T.-S. Zhang, "Knowledge graph and knowledge reasoning: A systematic review," *Journal of Electronic Science and Technology*, vol. 20, no. 2, p. 100159, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1674862X2200012X

[19] R. Abboud, I. Ceylan, T. Lukasiewicz, and T. Salvatori, "Boxe: A box embedding model for knowledge base completion," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9649–9661, 2020.

[20] T. A. Henzinger, "The temporal specification and verification of real-time systems," Ph.D. dissertation, Stanford University, USA, 1991. [Online]. Available: https://searchworks.stanford.edu/view/2148147

[21] ScienceDirect, "Jaccard Similarity," https://www.sciencedirect.com/topics/computer-science/jaccard-similarity, 2023.

[22] G. Attardi, "Wikiextractor," https://github.com/attardi/wikiextractor, 2015.

[23] E. Prud'hommeaux and A. Seaborne, "SPARQL Query Language for RDF - W3C recommendation." https://www.w3.org/TR/rdf-sparql-query/, 2018.

[24] J. Wielemaker, Z. Huang, and L. van der Meij, "Swi-prolog and the web," *Theory Pract. Log. Program.*, vol. 8, no. 3, pp. 363–392, 2008. [Online]. Available: https://doi.org/10.1017/S1471068407003237

[25] OpenAI, "Gpt-4 technical report," *ArXiv*, vol. abs/2303.08774, 2023.

[26] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[27] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[28] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[29] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Rozière, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. M. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, and et al., "The llama 3 herd of models," *CoRR*, vol. abs/2407.21783, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2407.21783

[30] G. Angeli, M. J. Johnson Premkumar, and C. D. Manning, "Leveraging linguistic structure for open domain information extraction," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, C. Zong and M. Strube, Eds. Beijing, China: Association for Computational Linguistics, Jul. 2015, pp. 344–354. [Online]. Available: https://aclanthology.org/P15-1034

[31] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.

[32] P.-S. Laplace, *A Philosophical Essay on Probabilities*. New York: Dover Publications, 1951, originally published in 1814 as "Essai Philosophique sur les Probabilités".

[33] G. A. Miller, "Wordnet: A lexical database for english," *Commun. ACM*, vol. 38, no. 11, p. 39–41, nov 1995. [Online]. Available: https://doi.org/10.1145/219717.219748

[34] Y. Qiu, Z. Zhao, Y. Ziser, A. Korhonen, E. M. Ponti, and S. B. Cohen, "Are large language models temporally grounded?" *arXiv preprint arXiv:2311.08398*, 2023.

[35] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's song in the ai ocean: A survey on hallucination in large language models," *arXiv preprint arXiv:2309.01219*, 2023.

[36] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "Factscore: Fine-grained atomic evaluation of factual precision in long form text generation," *arXiv preprint arXiv:2305.14251*, 2023.

[37] S. Han, H. Schoelkopf, Y. Zhao, Z. Qi, M. Riddell, L. Benson, L. Sun, E. Zubova, Y. Qiao, M. Burtell, D. Peng, J. Fan, Y. Liu, B. Wong, M. Sailor, A. Ni, L. Nan, J. Kasai, T. Yu, R. Zhang,

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3731191

S. R. Joty, A. R. Fabbri, W. Kryscinski, X. V. Lin, C. Xiong, and D. Radev, "FOLIO: natural language reasoning with first-order logic," *CoRR*, vol. abs/2209.00840, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2209.00840

[38] Z. Yang, L. Dong, X. Du, H. Cheng, E. Cambria, X. Liu, J. Gao, and F. Wei, "Language models as inductive reasoners," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, Y. Graham and M. Purver, Eds. St. Julian's, Malta: Association for Computational Linguistics, Mar. 2024, pp. 209–225. [Online]. Available: https://aclanthology.org/2024.eacl-long.13

[39] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," *arXiv preprint arXiv:1809.09600*, 2018.

[40] S. Moon, P. Shah, A. Kumar, and R. Subba, "Opendialkg: Explainable conversational reasoning with attention-based walks over knowledge graphs," in *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019, pp. 845–854.

[41] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," *arXiv preprint arXiv:1704.04368*, 2017.

[42] Z. Dong, T. Tang, J. Li, W. X. Zhao, and J.-R. Wen, "Bamboo: A comprehensive benchmark for evaluating long text modeling capacities of large language models," *arXiv preprint arXiv:2309.13345*, 2023.

[43] L. K. Umapathi, A. Pal, and M. Sankarasubbu, "Med-halt: Medical domain hallucination test for large language models," *arXiv preprint arXiv:2307.15343*, 2023.

[44] H. Kang and X.-Y. Liu, "Deficiency of large language models in finance: An empirical examination of hallucination," *arXiv preprint arXiv:2311.15548*, 2023.

[45] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," *arXiv preprint arXiv:2305.20050*, 2023.

[46] N. Varshney, W. Yao, H. Zhang, J. Chen, and D. Yu, "A stitch in time saves nine: Detecting and mitigating hallucinations of llms by validating low-confidence generation," *arXiv preprint arXiv:2307.03987*, 2023.

[47] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "Critic: Large language models can self-correct with tool-interactive critiquing," *arXiv preprint arXiv:2305.11738*, 2023.

[48] T. Vu, M. Iyyer, X. Wang, N. Constant, J. Wei, J. Wei, C. Tar, Y.-H. Sung, D. Zhou, Q. Le *et al.*, "Freshllms: Refreshing large language models with search engine augmentation," *arXiv preprint arXiv:2310.03214*, 2023.

[49] S. Yin, C. Fu, S. Zhao, T. Xu, H. Wang, D. Sui, Y. Shen, K. Li, X. Sun, and E. Chen, "Woodpecker: Hallucination correction for multimodal large language models," *arXiv preprint arXiv:2310.16045*, 2023.

[50] L. Chen, S. Li, J. Yan, H. Wang, K. Gunaratna, V. Yadav, Z. Tang, V. Srinivasan, T. Zhou, H. Huang *et al.*, "Alpagasus: Training a better alpaca with fewer data," *arXiv preprint arXiv:2307.08701*, 2023.

[51] M. Elaraby, M. Lu, J. Dunn, X. Zhang, Y. Wang, and S. Liu, "Halo: Estimation and reduction of hallucinations in open-source weak large language models," *arXiv preprint arXiv:2308.11764*, 2023.

[52] K. Tian, E. Mitchell, H. Yao, C. D. Manning, and C. Finn, "Fine-tuning language models for factuality," *arXiv preprint arXiv:2311.08401*, 2023.

[53] A. Azaria and T. Mitchell, "The internal state of an llm knows when its lying," *arXiv preprint arXiv:2304.13734*, 2023.

[54] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, "Inference-time intervention: Eliciting truthful answers from a language model," *arXiv preprint arXiv:2306.03341*, 2023.

[55] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A.-K. Dombrowski *et al.*, "Representation engineering: A top-down approach to ai transparency," *arXiv preprint arXiv:2310.01405*, 2023.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.